



INSTITUT NATIONAL
DES SCIENCES
APPLIQUÉES
CENTRE VAL DE LOIRE

Rapport Déploiement de VM Automatisé

LOPEZ Niels

Département STI, 5A ASL-IA

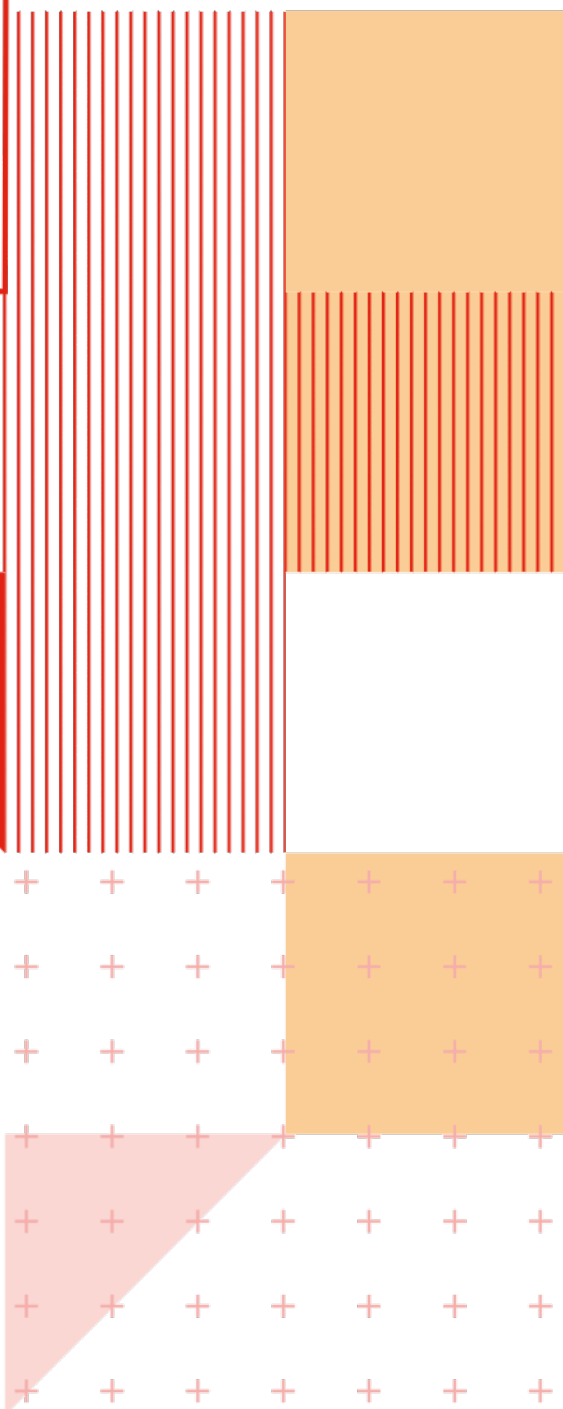


Table des matières

1. Premier déploiement avec TerraForm	3
1.1. Installation de Terraform	3
1.2. Création d'un user + token	3
1.3. Génération des fichiers de configuration Terraform	4
1.4. Terraform init et plan	5
1.5. Terraform apply	8
1.6. Terraform destroy	11
2. Déploiement avec adresse MAC fixée	13
2.1. DNSMasq to KEA & Unbound	13
2.2. Génération d'une adresse MAC statique	15
2.3. Attribution d'une IP statique à l'adresse MAC de la VM clonée	16
2.4. Terraform plan	17
2.5. Terraform apply	19
2.6. Terraform destroy	22
3. Implémentation de Cloud-init	23
3.1. Activation snippets dans local	23
3.2. Création d'un yaml pour Cloud-init	23
3.3. Gestion de l'envoi du yaml dans local:snippets	24
3.4. Dernières modifications du main.tf	25
3.5. Terraform plan	26
3.6. Terraform apply	29
3.7. Terraform destroy	33
4. Cleanup de l'espace de travail	37
5. Exercice final : une plateforme WordPress	41
5.1. Implémentation de WordPress avec nginx	42
5.2. Duplication de la VM	51
5.3. Nouvelle VM MariaDB	55
5.4. Nouvelle VM proxy HAProxy	60
6. GitHub	72

1. Premier déploiement avec TerraForm

1.1. Installation de Terraform

Le déploiement de VM automatique se fera avec Terraform, qu'il convient d'installer. Après avoir récupéré le binaire depuis hashicorp.com, il faut déplacer l'exécutable dans le dossier adéquat, et vérifier le bon fonctionnement de Terraform (test version):

```
user@debian:~/Téléchargements$ unzip terraform_1.16.2_linux_amd64.zip
Archive:  terraform_1.16.2_linux_amd64.zip
replace LICENSE.txt? [y]es, [n]o, [A]ll, [N]one, [r]ename: y
  inflating: LICENSE.txt
  inflating: terraform
user@debian:~/Téléchargements$ sudo mv terraform /usr/local/bin/
user@debian:~/Téléchargements$ terraform version
Terraform v1.16.2
on linux_amd64
```

1.2. Création d'un user + token

Après avoir créé, dans le CLI du proxmox, un user PVE

```
1 root@pve:~# pveum user add terraform@pve --password userTerra
```

Il faut lui ajouter les bons droits, listés dans le role « TerraRole » créé à cet effet (et modifié via l'interface Proxmox, plus simple d'utilisation qu'en CLI)

```
1 root@pve:~# pveum role add TerraRole -privs "Datastore.AllocateSpace Datastore.Audit
Pool.Allocate Sys.Audit Sys.Console Sys.Modify VM.Allocate VM.Audit VM.Clone VM.Config.CDRom
VM.Config.Cloudinit VM.Config.CPU VM.Config.Disk VM.Config.HWType VM.Config.Memory
VM.Config.Network VM.Config.Options VM.Migrate VM.PowerMgmt SDN.Use"
```

Roles	Yes	PVESUNUser	SDN.Audit SDN.Use
	Yes	PVESysAdmin	Sys.Audit Sys.Console Sys.Syslog
	Yes	PVETemplateUser	VM.Audit VM.Clone
	Yes	PVEUserAdmin	Group.Allocate Realm.AllocateUser User.Modify
	Yes	PVEVMAdmin	VM.Allocate VM.Audit VM.Backup VM.Clone VM.Config.CDRom VM.Config.CPU VM.Config.Cloudinit VM.Config.Disk VM.Config.HWType VM.Config.Memory VM.Config.Network VM.Config.Pool.Allocate VM.Config.SDN VM.Config.Snapshot VM.Config.Storage VM.Config.VirtIO VM.Config.VirtIO.Scsi VM.Config.VirtIO.Scsi.Snapshot VM.Config.VirtIO.Scsi.Snapshot.Rollback VM.Migrate VM.PowerMgmt VM.Replication VM.Snapshot VM.Snapshot.Rollback
Options	Yes	PVEVMUser	VM.Audit VM.Backup VM.Config.CDRom VM.Config.Cloudinit VM.Console VM.GuestAgent.Audit VM.GuestAgent.FileRead VM.GuestAgent.FileSystemMgmt VM.GuestAgent.FileWrite VM.P
	No	TerraRole	Datastore.AllocateSpace Datastore.Audit Pool.Allocate SDN.Use Sys.Audit Sys.Console Sys.Modify VM.Allocate VM.Audit VM.Clone VM.Config.CDRom VM.Config.CPU VM.Config.Cloudini

Donc, lui ajouter le rôle, et générer le token associé à cet user:

```
root@pve:~# pveum acl modify / -user terraform@pve -role TerraRole
root@pve:~# pveum user token add terraform@pve tf-token --privsep 0
```

key	value
full-tokenid	terraform@pve!tf-token
info	{"privsep": "0"}
value	2ef5d121-2f70-48c7-a17b-78e7dc22d76e

```
root@pve:~#
```

1.3. Génération des fichiers de configuration Terraform

Les détails de la configuration sont stockés dans le fichier « main.tf »:

```
1 user@debian:~/tform$ cat main.tf
2 provider "proxmox" {
3   pm_api_url      = "https://192.168.122.220:8006/api2/json"
4   pm_api_token_id  = "terraform@pve!tf-token"
5   pm_api_token_secret = "2ef5d121-2f70-48c7-a17b-78e7dc22d76e"
6   pm_tls_insecure   = true
7   pm_minimum_permission_check = false
8 }
9
10 terraform {
11   required_providers {
12     proxmox = {
13       source = "telmate/proxmox"
14       version = "3.0.2-rc07"
15     }
16   }
17 }
18
19 resource "proxmox_vm_qemu" "debian_clone" {
20   name          = "debian-TerraCloned"
21   target_node   = "pve"
22
23   # os_type      = "cloud-init"
24   clone          = "debian"
25   full_clone     = true
26   agent          = 1
27   start_at_node_boot = true
28
29   vmid          = 120
30   memory        = 2048
31
32   cpu {
33     cores    = 2
34     sockets  = 1
35     type     = "host"
36   }
37
38   disk {
39     slot     = "scsi0"
40     size     = "40G"
41     type     = "disk"
42     storage  = "local-lvm"
43   }
44
45   network {
46     id       = 0
47     model    = "virtio"
```

```
48     bridge     = "vmbr1"
49   }
50
51 }
```

Les caractéristiques de la VM sont passés en paramètre suivant les consignes. Le seul provider nécessaire à cette étape est Telmate.

1.4. Terraform init et plan

```
user@debian:~/tform$ terraform init -upgrade
Initializing the backend...
```

Initializing provider plugins...

```
- Finding telmate/proxmox versions matching "3.0.2-rc07"...
- Installing telmate/proxmox v3.0.2-rc07...
- Installed telmate/proxmox v3.0.2-rc07 (self-signed, key ID A9EBBE091B35AFCE)
Partner and community providers are signed by their developers.
If you'd like to know more about provider signing, you can read about it here:
https://developer.hashicorp.com/terraform/cli/plugins/signing
```

Terraform has made some changes to the provider dependency selections recorded in the .terraform.lock.hcl file. Review those changes and commit them to your version control system if they represent changes you intended to make.

Terraform has been successfully initialized!

You may now begin working with Terraform. Try running "terraform plan" to see any changes that are required for your infrastructure. All Terraform commands should now work.

If you ever set or change modules or backend configuration for Terraform, rerun this command to reinitialize your working directory. If you forget, other commands will detect it and remind you to do so if necessary.

```
user@debian:~/tform$
```

```
1 user@debian:~/tform$ terraform plan
2
3 Terraform used the selected providers to generate the following execution plan.
4 Resource actions are indicated with the following symbols:
5   + create
6
7 Terraform will perform the following actions:
8
9   # proxmox_vm_qemu.debian_clone will be created
10  + resource "proxmox_vm_qemu" "debian_clone" {
11    + additional_wait      = 5
12    + agent                = 1
13    + agent_timeout       = 90
14    + automatic_reboot    = true
15    + automatic_reboot_severity = "error"
16    + balloon             = 0
17    + bios                = "seabios"
```

```

18      + boot                = (known after apply)
19      + bootdisk             = (known after apply)
20      + ciupgrade             = false
21      + clone                 = "debian"
22      + clone_wait            = 10
23      + current_node          = (known after apply)
24      + default_ipv4_address  = (known after apply)
25      + default_ipv6_address  = (known after apply)
26      + define_connection_info = true
27      + description           = "Managed by Terraform."
28      + force_create           = false
29      + full_clone             = true
30      + hotplug                = "network,disk,usb"
31      + id                    = (known after apply)
32      + kvm                    = true
33      + linked_vmid            = (known after apply)
34      + memory                 = 2048
35      + name                   = "debian-TerraCloned"
36      + protection             = false
37      + reboot_required        = (known after apply)
38      + scsihw                 = "lsi"
39      + skip_ipv4              = false
40      + skip_ipv6              = false
41      + ssh_host               = (known after apply)
42      + ssh_port               = (known after apply)
43      + start_at_node_boot     = true
44      + tablet                 = true
45      + target_node            = "pve"
46      + unused_disk            = (known after apply)
47      + vm_state               = "running"
48      + vmid                   = 120
49
50      + cpu {
51          + cores    = 2
52          + limit    = 0
53          + numa     = false
54          + sockets  = 1
55          + type     = "host"
56          + units    = 0
57          + vcores   = 0
58      }
59
60      + disk {
61          + backup      = true
62          + id           = (known after apply)
63          + iops_r_burst = 0
64          + iops_r_burst_length = 0
65          + iops_r_concurrent = 0
66          + iops_wr_burst = 0

```

```
67     + iops_wr_burst_length = 0
68     + iops_wr_concurrent   = 0
69     + linked_disk_id       = (known after apply)
70     + mbps_r_burst         = 0
71     + mbps_r_concurrent    = 0
72     + mbps_wr_burst        = 0
73     + mbps_wr_concurrent   = 0
74     + passthrough          = false
75     + size                  = "40G"
76     + slot                  = "scsi0"
77     + storage               = "local-lvm"
78     + type                  = "disk"
79   }
80
81   + network {
82     + bridge    = "vmbri1"
83     + firewall  = false
84     + id        = 0
85     + link_down = false
86     + macaddr   = (known after apply)
87     + model     = "virtio"
88   }
89
90   + smbios (known after apply)
91 }
92
93 Plan: 1 to add, 0 to change, 0 to destroy.
94
95 _____
96
97 Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to
98 take exactly these actions if you run "terraform apply" now.
```

1.5. Terraform apply

```
user@debian:~/tform$ terraform apply
```

Terraform used the selected providers to generate the following execution plan.
Resource actions are indicated with the following symbols:

```
+ create
```

Terraform will perform the following actions:

```
# proxmox_vm_qemu.debian_clone will be created
+ resource "proxmox_vm_qemu" "debian_clone" {
  + additional_wait      = 5
  + agent                = 1
  + agent_timeout        = 90
  + automatic_reboot     = true
  + automatic_reboot_severity = "error"
  + balloon              = 0
  + bios                 = "seabios"
  + boot                 = (known after apply)
  + bootdisk             = (known after apply)
  + ciupgrade            = false
  + clone                = "debian"
  + clone_wait           = 10
  + current_node          = (known after apply)
  + default_ipv4_address = (known after apply)
  + default_ipv6_address = (known after apply)
  + define_connection_info = true
  + description          = "Managed by Terraform."
  + force_create         = false
  + full_clone           = true
  + hotplug              = "network,disk,usb"
  + id                   = (known after apply)
  + kvm                  = true
  + linked_vmid          = (known after apply)
  + memory               = 2048
  + name                 = "debian-TerraCloned"
  + protection           = false
  + reboot_required      = (known after apply)
  + scsihw               = "lsi"
  + skip_ipv4            = false
  + skip_ipv6            = false
  + ssh_host             = (known after apply)
  + ssh_port             = (known after apply)
  + start_at_node_boot   = true
  + tablet               = true
  + target_node          = "pve"
  + unused_disk          = (known after apply)
  + vm_state             = "running"
  + vmid                 = 120

  + cpu {
```

```
+ cores      = 2
+ limit      = 0
+ numa       = false
+ sockets    = 1
+ type       = "host"
+ units      = 0
+ vcores     = 0
}

+ disk {
  + backup          = true
  + id              = (known after apply)
  + iops_r_burst     = 0
  + iops_r_burst_length = 0
  + iops_r_concurrent = 0
  + iops_wr_burst     = 0
  + iops_wr_burst_length = 0
  + iops_wr_concurrent = 0
  + linked_disk_id    = (known after apply)
  + mbps_r_burst      = 0
  + mbps_r_concurrent = 0
  + mbps_wr_burst      = 0
  + mbps_wr_concurrent = 0
  + passthrough       = false
  + size              = "40G"
  + slot              = "scsi0"
  + storage            = "local-lvm"
  + type              = "disk"
}

+ network {
  + bridge    = "vmbri"
  + firewall  = false
  + id        = 0
  + link_down = false
  + macaddr   = (known after apply)
  + model     = "virtio"
}

+ smbios (known after apply)
}
```

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

```
proxmox_vm_qemu.debian_clone: Creating...
proxmox_vm_qemu.debian_clone: Still creating... [00m10s elapsed]
proxmox_vm_qemu.debian_clone: Still creating... [00m20s elapsed]
proxmox_vm_qemu.debian_clone: Still creating... [00m30s elapsed]
proxmox_vm_qemu.debian_clone: Still creating... [00m40s elapsed]
proxmox_vm_qemu.debian_clone: Still creating... [00m50s elapsed]
proxmox_vm_qemu.debian_clone: Still creating... [01m00s elapsed]
proxmox_vm_qemu.debian_clone: Still creating... [01m10s elapsed]
proxmox_vm_qemu.debian_clone: Still creating... [01m20s elapsed]
proxmox_vm_qemu.debian_clone: Still creating... [01m30s elapsed]
proxmox_vm_qemu.debian_clone: Still creating... [01m40s elapsed]
proxmox_vm_qemu.debian_clone: Still creating... [01m50s elapsed]
proxmox_vm_qemu.debian_clone: Still creating... [02m00s elapsed]
proxmox_vm_qemu.debian_clone: Creation complete after 2m2s [id=pve/qemu/120]
```

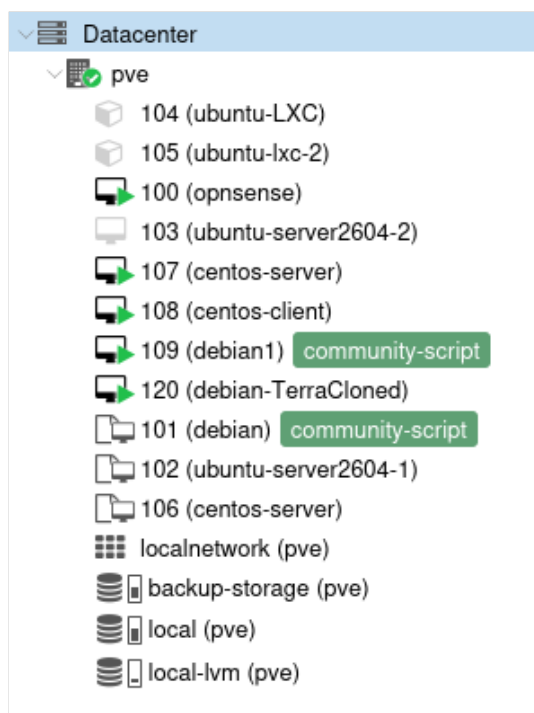
Warning: Qemu Guest Agent is enabled but no IPv6 address is found

```
with proxmox_vm_qemu.debian_clone,
on main.tf line 18, in resource "proxmox_vm_qemu" "debian_clone":
18: resource "proxmox_vm_qemu" "debian_clone" {
```

Qemu Guest Agent is enabled in your configuration but no IPv6 address was found before the time ran out, increasing 'agent_timeout' could resolve this issue.
To suppress this warning set 'skip_ipv6' to true.

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Et la VM créée (id 120):



1.6. Terraform destroy

```
user@debian:~/tform$ terraform destroy
proxmox_vm_qemu.debian_clone: Refreshing state... [id=pve/qemu/120]
```

Terraform used the selected providers to generate the following execution plan.
Resource actions are indicated with the following symbols:

- destroy

Terraform will perform the following actions:

```
# proxmox_vm_qemu.debian_clone will be destroyed
- resource "proxmox_vm_qemu" "debian_clone" {
  - additional_wait      = 5 -> null
  - agent               = 1 -> null
  - agent_timeout       = 90 -> null
  - automatic_reboot    = true -> null
  - automatic_reboot_severity = "error" -> null
  - balloon             = 0 -> null
  - bios                = "seabios" -> null
  - boot                = "order=scsi0" -> null
  - ciupgrade           = false -> null
  - clone               = "debian" -> null
  - clone_wait          = 10 -> null
  - cores               = 0 -> null
  - current_node        = "pve" -> null
  - default_ipv4_address = "10.0.0.68" -> null
  - define_connection_info = true -> null
  - description         = "Managed by Terraform." -> null
  - force_create        = false -> null
  - full_clone          = true -> null
  - hotplug             = "network,disk,usb" -> null
  - id                  = "pve/qemu/120" -> null
  - kvm                 = true -> null
  - linked_vmid         = 0 -> null
  - memory              = 2048 -> null
  - name                = "debian-TerraCloned" -> null
  - numa               = false -> null
  - protection          = false -> null
  - qemu_os             = "l26" -> null
  - reboot_required     = false -> null
  - scsihw              = "lsi" -> null
  - skip_ipv4           = false -> null
  - skip_ipv6           = false -> null
  - sockets             = 0 -> null
  - ssh_host            = "10.0.0.68" -> null
  - ssh_port            = "22" -> null
  - start_at_node_boot  = true -> null
  - tablet              = true -> null
  - tags                = null
  - target_node         = "pve" -> null
```

```
- unused_disk      = [] -> null
- vcpus            = 0 -> null
- vm_state         = "running" -> null
- vmid             = 120 -> null
# (10 unchanged attributes hidden)

- cpu {
  - cores      = 2 -> null
  - limit      = 0 -> null
  - numa       = false -> null
  - sockets    = 1 -> null
  - type       = "host" -> null
  - units      = 0 -> null
  - vcores     = 0 -> null
  # (1 unchanged attribute hidden)
}

- disk {
  - backup      = true -> null
  - discard     = false -> null
  - emulattedsd = false -> null
  - format      = "raw" -> null
  - id          = 1 -> null
  - iops_r_burst = 0 -> null
  - iops_r_burst_length = 0 -> null
  - iops_r_concurrent = 0 -> null
  - iops_wr_burst = 0 -> null
  - iops_wr_burst_length = 0 -> null
  - iops_wr_concurrent = 0 -> null
  - iothread    = false -> null
  - linked_disk_id = -1 -> null
  - mbsps_r_burst = 0 -> null
  - mbsps_r_concurrent = 0 -> null
  - mbsps_wr_burst = 0 -> null
  - mbsps_wr_concurrent = 0 -> null
  - passthrough  = false -> null
  - readonly     = false -> null
  - replicate    = false -> null
  - size         = "40G" -> null
  - slot         = "scsi0" -> null
  - storage      = "local-lvm" -> null
  - type         = "disk" -> null
  # (6 unchanged attributes hidden)
}

- network {
  - bridge      = "vmbri1" -> null
  - firewall    = false -> null
  - id          = 0 -> null
  - link_down   = false -> null
  - macaddr     = "bc:24:11:a4:aa:43" -> null
  - model       = "virtio" -> null
  - mtu         = 0 -> null
  - queues      = 0 -> null
  - rate        = 0 -> null
  - tag         = 0 -> null
}

- smbios {
  - uuid        = "b455933a-67c2-4bb3-943e-7ff6f2d19300" -> null
  # (6 unchanged attributes hidden)
}

- startup_shutdown {
  - order       = -1 -> null
  - shutdown_timeout = -1 -> null
  - startup_delay = -1 -> null
}
}
```

Plan: 0 to add, 0 to change, 1 to destroy.

Do you really want to destroy all resources?

Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

proxmox_vm_qemu.debian_clone: Destroying... [id=pve/qemu/120]
proxmox_vm_qemu.debian_clone: Destruction complete after 4s

Destroy complete! Resources: 1 destroyed.

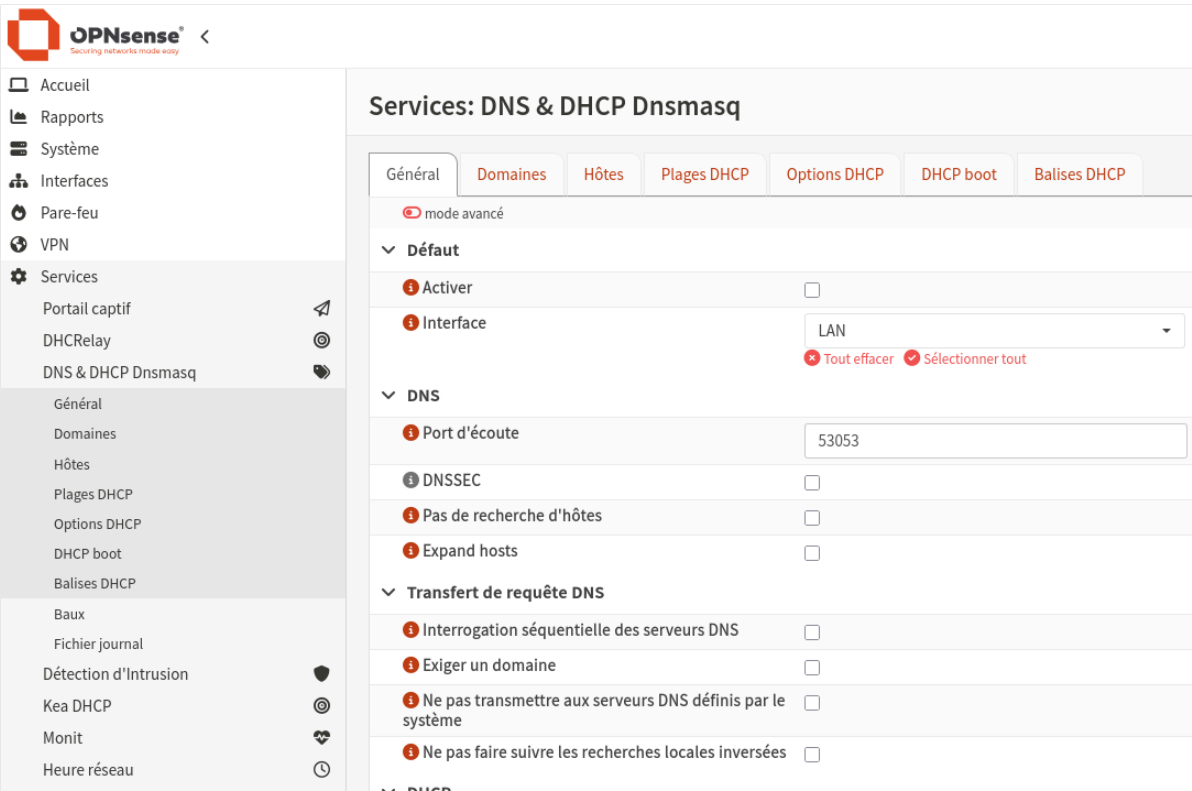
user@debian:~/terraform

Cela fonctionne parfaitement.

2. Déploiement avec adresse MAC fixée

2.1. DNSMasq to KEA & Unbound

Arrêt du DNSMasq:



OPNsense® Securing networks made easy

Accueil
Rapports
Système
Interfaces
Pare-feu
VPN
Services
Portail captif
DHCRelay
DNS & DHCP Dnsmasq
Général
Domaines
Hôtes
Plages DHCP
Options DHCP
DHCP boot
Balises DHCP
Baux
Fichier journal
Détection d'intrusion
Kea DHCP
Monit
Heure réseau

Services: DNS & DHCP Dnsmasq

Général | Domaines | Hôtes | Plages DHCP | Options DHCP | DHCP boot | Balises DHCP

mode avancé

▼ Défaut

Activer ☐

Interface LAN
Tout effacer Sélectionner tout

▼ DNS

Port d'écoute 53053

DNSSEC ☐

Pas de recherche d'hôtes ☐

Expand hosts ☐

▼ Transfert de requête DNS

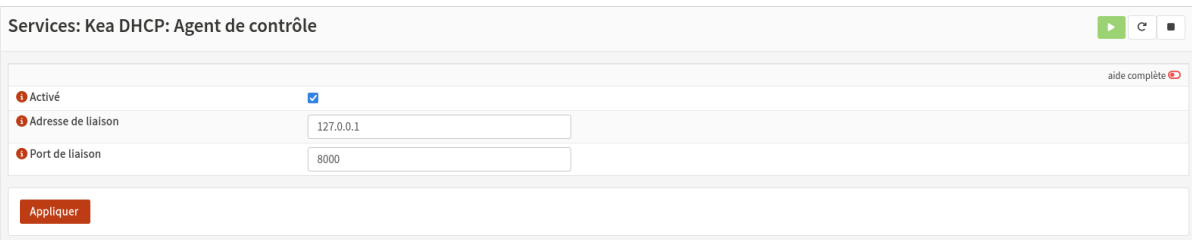
Interrogation séquentielle des serveurs DNS ☐

Exiger un domaine ☐

Ne pas transmettre aux serveurs DNS définis par le système ☐

Ne pas faire suivre les recherches locales inversées ☐

Activation de KEA Agent:



Services: Kea DHCP: Agent de contrôle

aide complète

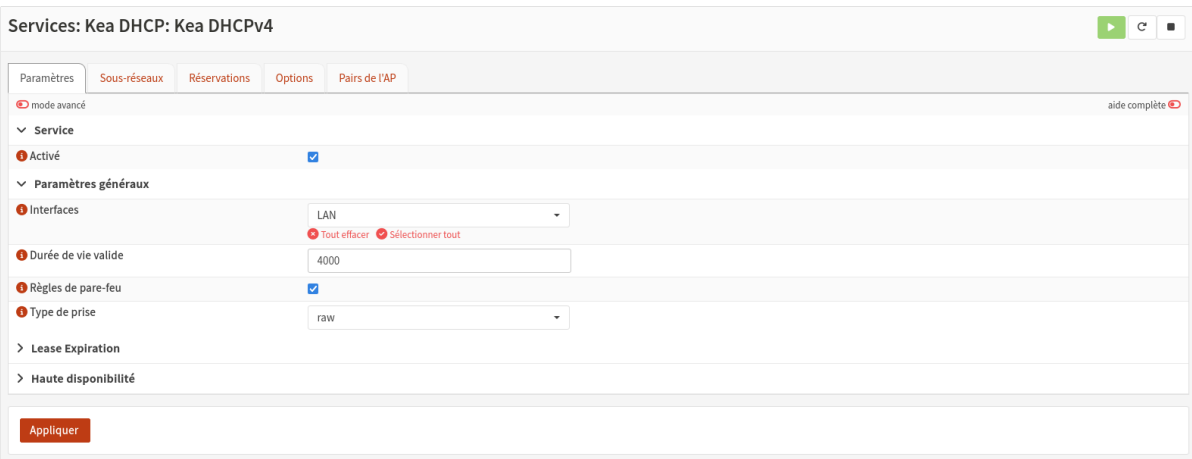
Activé ☒

Adresse de liaison 127.0.0.1

Port de liaison 8000

Appliquer

Activation de KEA DHCP:



Services: Kea DHCP: Kea DHCPv4

Paramètres | Sous-réseaux | Réservations | Options | Pairs de l'AP

mode avancé

aide complète

▼ Service

Activé ☒

▼ Paramètres généraux

Interfaces LAN
Tout effacer Sélectionner tout

Durée de vie valide 4000

Règles de pare-feu ☒

Type de prise raw

> Lease Expiration

> Haute disponibilité

Appliquer

Définition du sous-réseau visé:

Services: Kea DHCP: Kea DHCPv4

Paramètres Sous-réseaux Réservations Options Pairs de l'AP

Recherche

50

☐	Sous-réseau	Description	Piscines	Actions
☐	10.0.0.0/24	LAN	10.0.0.41 - 10.0.0.245	

<

1

>

Affichage des entrées 1 à 1 sur 1

Appliquer

Activation du service Unbound:

Services: Unbound DNS: Général

mode avancé aide complète

1 Activer Unbound

☒

2 Port d'écoute

53

3 Interfaces réseau

Tout (recommandé)

Tout effacer Sélectionner tout

4 Activer le support DNSSEC

☐

5 Activer le support DNS64

☐

6 Préfixe DNS64

64:ff9b::/96

7 Activer le mode AAAA uniquement

☐

8 Enregistrer les baux DHCP4 ICS

☐

9 Contournement de domaine DHCP

10 Enregistrer les correspondances statiques DHCP

☐

11 Ne pas enregistrer les adresses IPv6 Link-Local

☐

12 Ne pas enregistrer les enregistrements du système A/AAAA

☐

13 Prise en charge des commentaires TXT

☐

14 Vider le cache DNS lors du rechargement

☐

15 Forcer SafeSearch

☐

16 Type de zone locale

transparent

Appliquer

Allocation dynamique d'IP fonctionnelle avec KEA (machine redémarrée):

ubuntu-server2604-2 (Uptime: 00:01:18)

Status

running

HA State

none

Node

pve

CPU usage

2.75% of 1 CPU

Memory usage

14.73% (603.36 MiB of 4.00 GiB)

Host memory usage

1.19 GiB

Bootdisk size

32.00 GiB

IPs

10.0.0.41
fe80::be24:11ff:fe25:a21b

More

Définition dynamique du DNS et Gateway:

Services: Kea DHCP: Kea DHCPv4						
ParamètresSous-réseauxRéservationsOptionsPairs de l'AP						
Recherche 50						
☐	Description	Match Code	Match Data	Set Code	Set Data	Actions
☐	dns			name server [5]	10.0.0.1	  
☐	gateway			router [3]	10.0.0.1	  
« 1 »						Affichage des entrées 1 à 2 sur 2 

2.2. Génération d'une adresse MAC statique

Ajout d'Ivoronin comme provider et génération de la ressource MAC address:

```
main.tf
1 terraform {
2   required_providers {
3     proxmox = {
4       source = "telmate/proxmox"
5       version = "3.0.2-rc04" #3.0.2-rc07
6     }
7     macaddress = {
8       source = "ivoronin/macaddress"
9       version = "0.3.0"
10    }
11  }
12
13  [...]
14
15  resource "macaddress" "vmmac" {
16  }
```

Puis, utilisation de cette adresse MAC dans les spécifications réseau du clone:

```
main.tf
1 network {
2   id      = 0
3   model   = "virtio"
4   bridge  = "vmbri1"
5   macaddr = upper(macaddress.vmmac.address)
6 }
```

En uppercase suivant les exigences de proxmox.

2.3. Attribution d'une IP statique à l'adresse MAC de la VM clonée

Pour cela il faut créer un group + user pour récupérer un token d'accès API à Opnsense:

Système: Accès: Groupes

Recherche						
☐	Nom du groupe	Privilèges	Membres	Description	Réseaux source	Actions
☐	admins	All pages	root	System Administrators		 
☐	tf-group	Services: DHCP: Kea Ctrl Age...	terraform			 

« 1 » Affichage des entrées 1 à 2 sur 2 

avec les privilèges de contrôle de KEA. Puis, attribuer ce rôle à un user pour le quel on pourra récupérer une clé API:

Système: Accès: Utilisateurs

Utilisateurs Clés Api

Recherche				
☐	Nom d'utilisateur	Membre du groupe	Nom complet	Actions
☐	 root	admins	System Administrator	   
☐	 terraform	tf-group		   

« 1 » Affichage des entrées 1 à 2 sur 2    

Système: Accès: Utilisateurs

Utilisateurs

Clés Api

Q

Recherche

↺

50

☰

🔗

🗨

☐	Nom d'utilisateur	Clé Api	Actions
☐	terraform	AncmTmUYXI5csabprglFjDahbfRohVt7jYNm+UbKnPwAApQXaHXV7K3/...	🗑

«

◀

1

▶

»

Affichage des entrées 1 à 1 sur 1

🗑

Cette clé peut-être ajoutée dans le provider Browningluke qui s'interface avec Opnsense:

main.tf

```
1 provider "opnsense" {
2   uri = "https://192.168.122.197"
3   api_key = "AncmTmUYXl5csabprgylFjDahbfRohVt7jYNm+UbKnPwAApQXaHXV7K3/UeYLUkcFY>
4   api_secret = "JU4/5kxzlFm5tALdiscA5Bzt1s6F+TZh6GoY2JLf0ITYUVadoVs4CjEq8lGH2kv>
5   allow_insecure = true
6 }
```

avec allow_insecure pour contourner l'autovalidation du certificat.

puis:

main.tf

```
1 resource "opnsense_kea_dhcpv4_reservation" "debian_clone" {
2   subnet_id   = "4f275fc3-7f2d-4bdb-b8a7-eb794075aa7f"
3   ip_address  = "10.0.0.140"
4   mac_address = lower(macaddress.vmmac.address)
5   hostname    = "tf_cloned_debian"
6   description = "Static IP for tf_cloned_debian"
7 }
```

Le subnet ID correspond à l'UUID du sous réseau créé précédemment de plage 10.0.0.41 - 10.0.0.245. L'adresse IP fixe attribuée à l'adresse MAC est 10.0.0.140 et l'adresse mac est passée en lowercase (contrairement à OpnSense qui utilise l'Uppercase...)

2.4. Terraform plan

```
Destroy complete! Resources: 3 destroyed.
user@debian:~/terraform$ terraform plan
```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

+ create

Terraform will perform the following actions:

```
# macaddress.vmmac will be created
+ resource "macaddress" "vmmac" {
+   address = (known after apply)
+   id      = (known after apply)
+ }

# opnsense_kea_dhcpv4_reservation.debian_clone will be created
+ resource "opnsense_kea_dhcpv4_reservation" "debian_clone" {
+   description = "Static IP for tf_cloned_debian"
+   hostname    = "tf_cloned_debian"
+   id          = (known after apply)
+   ip_address  = "10.0.0.140"
+   mac_address = (known after apply)
+   subnet_id   = "4f275fc3-7f2d-4bdb-b8a7-eb794075aa7f"
+ }

# proxmox_vm_qemu.debian_clone will be created
+ resource "proxmox_vm_qemu" "debian_clone" {
+   additional_wait = 5
+   agent           = 1
+   agent_timeout   = 90
+   automatic_reboot = true
+   balloon         = 0
+   bios            = "seabios"
+   boot            = (known after apply)
+   bootdisk        = (known after apply)
+   ciupgrade       = false
+   clone           = "debian"
+   clone_wait      = 10
+   current_node     = (known after apply)
+   default_ipv4_address = (known after apply)
+   default_ipv6_address = (known after apply)
+   define_connection_info = true
+   description      = "Managed by Terraform."
+   force_create      = false
+   full_clone       = true
+   hotplug          = "network,disk,usb"
+   id               = (known after apply)
+   kvm              = true
```

```
+ linked_vmid      = (known after apply)
+ memory          = 2048
+ name            = "debian-TerraCloned"
+ onboot          = true
+ protection       = false
+ reboot_required = (known after apply)
+ scsihw          = "lsi"
+ skip_ipv4        = false
+ skip_ipv6        = false
+ ssh_host        = (known after apply)
+ ssh_port        = (known after apply)
+ tablet          = true
+ target_node     = "pve"
+ unused_disk     = (known after apply)
+ vm_state        = "running"
+ vmid            = 120

+ cpu {
  + cores   = 2
  + limit   = 0
  + numa    = false
  + sockets = 1
  + type    = "host"
  + units   = 0
  + vcores  = 0
}

+ disk {
  + backup           = true
  + id               = (known after apply)
  + iops_r_burst     = 0
  + iops_r_burst_length = 0
  + iops_r_concurrent = 0
  + iops_wr_burst    = 0
  + iops_wr_burst_length = 0
  + iops_wr_concurrent = 0
  + linked_disk_id   = (known after apply)
  + mbytes_r_burst   = 0
  + mbytes_r_concurrent = 0
  + mbytes_wr_burst  = 0
  + mbytes_wr_concurrent = 0
  + passthrough      = false
  + size             = "40G"
  + slot             = "scsi0"
  + storage          = "local-lvm"
  + type             = "disk"
}

+ network {
  + bridge = "vmbr1"
  + firewall = false
  + id      = 0
  + link_down = false
  + macaddr = (known after apply)
  + model   = "virtio"
}

+ smbios (known after apply)
}
```

Plan: 3 to add, 0 to change, 0 to destroy.

Note: You didn't use the `-out` option to save this plan, so Terraform can't guarantee to take exactly these actions if you run `"terraform apply"` now.
`user@debian:~/tforms$`

2.5. Terraform apply

```
user@debian:~/terraform$ terraform apply
```

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:

```
+ create
```

Terraform will perform the following actions:

```
# macaddress.vmmac will be created
+ resource "macaddress" "vmmac" {
  + address = (known after apply)
  + id      = (known after apply)
}

# opnsense_kea_dhcpv4_reservation.debian_clone will be created
+ resource "opnsense_kea_dhcpv4_reservation" "debian_clone" {
  + description = "Static IP for tf cloned debian"
  + hostname    = "tf_cloned_debian"
  + id          = (known after apply)
  + ip_address  = "10.0.0.140"
  + mac_address = (known after apply)
  + subnet_id   = "4f275fc3-7f2d-4bdb-b8a7-eb794075aa7f"
}

# proxmox_vm_qemu.debian_clone will be created
+ resource "proxmox_vm_qemu" "debian_clone" {
  + additional_wait = 5
  + agent           = 1
  + agent_timeout   = 90
  + automatic_reboot = true
  + balloon         = 0
  + bios            = "seabios"
  + boot            = (known after apply)
  + bootdisk        = (known after apply)
  + ciupgrade       = false
  + clone           = "debian"
  + clone_wait      = 10
  + current_node    = (known after apply)
  + default_ipv4_address = (known after apply)
  + default_ipv6_address = (known after apply)
  + define_connection_info = true
  + description      = "Managed by Terraform."
  + force_create      = false
  + full_clone       = true
  + hotplug          = "network,disk,usb"
  + id               = (known after apply)
  + kvm              = true
  + linked_vmid      = (known after apply)
  + memory           = 2048
}
```

```
+ name           = "debian-TerraCloned"
+ onboot         = true
+ protection     = false
+ reboot_required = (known after apply)
+ scsihw         = "lsi"
+ skip_ipv4      = false
+ skip_ipv6      = false
+ ssh_host       = (known after apply)
+ ssh_port       = (known after apply)
+ tablet         = true
+ target_node    = "pve"
+ unused_disk    = (known after apply)
+ vm_state       = "running"
+ vmid           = 120
```

```
+ cpu {
+   cores    = 2
+   limit    = 0
+   numa     = false
+   sockets  = 1
+   type     = "host"
+   units    = 0
+   vcores   = 0
+ }
```

```
+ disk {
+   backup      = true
+   id          = (known after apply)
+   iops_r_burst = 0
+   iops_r_burst_length = 0
+   iops_r_concurrent = 0
+   iops_wr_burst = 0
+   iops_wr_burst_length = 0
+   iops_wr_concurrent = 0
+   linked_disk_id = (known after apply)
+   mbps_r_burst = 0
+   mbps_r_concurrent = 0
+   mbps_wr_burst = 0
+   mbps_wr_concurrent = 0
+   passthrough = false
+   size        = "40G"
+   slot        = "scsi0"
+   storage     = "local-lvm"
+   type        = "disk"
+ }
```

```
+ network {
+   bridge    = "vmbr1"
+   firewall  = false
+   id        = 0
+   link_down = false
```

```
+ macaddr = (known after apply)
+ model   = "virtio"
+ }
```

```
+ smbios (known after apply)
+ }
```

Plan: 3 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

```
macaddress.vmmac: Creating...
macaddress.vmmac: Creation complete after 0s [id=16:43:7e:85:3b:8c]
opnsense_kea_dhcpv4_reservation.debian_clone: Creating...
proxmox_vm_qemu.debian_clone: Creating...
opnsense_kea_dhcpv4_reservation.debian_clone: Creation complete after 0s [id=9d6ad7c3-0e26-4b9f-8e56-ad5271bc2399]
proxmox_vm_qemu.debian_clone: Still creating... [00m10s elapsed]
proxmox_vm_qemu.debian_clone: Still creating... [00m20s elapsed]
proxmox_vm_qemu.debian_clone: Still creating... [00m30s elapsed]
proxmox_vm_qemu.debian_clone: Still creating... [00m40s elapsed]
proxmox_vm_qemu.debian_clone: Still creating... [00m50s elapsed]
proxmox_vm_qemu.debian_clone: Still creating... [01m00s elapsed]
proxmox_vm_qemu.debian_clone: Still creating... [01m10s elapsed]
proxmox_vm_qemu.debian_clone: Still creating... [01m20s elapsed]
proxmox_vm_qemu.debian_clone: Still creating... [01m30s elapsed]
proxmox_vm_qemu.debian_clone: Still creating... [01m40s elapsed]
proxmox_vm_qemu.debian_clone: Still creating... [01m50s elapsed]
proxmox_vm_qemu.debian_clone: Still creating... [02m00s elapsed]
proxmox_vm_qemu.debian_clone: Creation complete after 2m4s [id=pve/qemu/120]
```

Warning: Qemu Guest Agent is enabled but no IPv6 address is found

```
with proxmox_vm_qemu.debian_clone,
on main.tf line 37, in resource "proxmox_vm_qemu" "debian_clone":
37: resource "proxmox_vm_qemu" "debian_clone" {
```

Qemu Guest Agent is enabled in your configuration but no IPv6 address was found before the time ran out, increasing 'agent_timeout' could resolve this issue. To suppress this warning set 'skip_ipv6' to true.

Apply complete! Resources: 3 added, 0 changed, 0 destroyed.

user@debian:~/tform\$

Il est possible d'ajouter le tag skip_ipv6 pour supprimer le warning qui pop à chaque Apply.
Sur proxmox:

Virtual Machine 120 (debian-TerraCloned) on node 'pve'No Tags

Summary

ConsoleHardwareCloud-InitOptionsTask HistoryMonitorBackupReplicationSnapshotsFirewallPermissions

debian-TerraCloned (Uptime: 00:02:54)

Statusrunning

HA Statenone

Nodepve

CPU usage0.66% of 2 CPUs

Memory usage21.85% (447.57 MiB of 2.00 GiB)

Host memory usage447.57 MiB

Bootdisk size40.00 GiB

IPs10.0.0.140
fe80::1443:7eff:fe85:3b8c

Notes

Managed by Ter

Network Information

Name	MAC address	IP address
lo	00:00:00:00:00:00	127.0.0.1/8 ::1/128
ens18	16:43:7e:85:3b:8c	10.0.0.140/24 fe80::1443:7eff:fe85:3b8c/64

On constate bien que l'adresse MAC générée (id=16:43:7e:85:3b:8c) a été correctement, attribuée, ainsi que l'adresse IP, confirmé par OpnSense:

Services: Kea DHCP: Kea DHCPv4

ParamètresSous-réseauxRéservationsOptionsPairs de l'AP

Recherche

50

☐	Sous-réseau	Adresse IP	Adresse MAC	ID Client	Nom d'hôte	Description	Actions
☒	10.0.0.0/24						
☐	10.0.0.0/24	10.0.0.140	16:43:7e:85:3b:8c		tf_cloned_debian	Static IP for tf_cloned_...	

«1»

Affichage des entrées 1 à 1 sur 1

Appliquer

21

2.6. Terraform destroy

```
Apply complete! Resources: 3 added, 0 changed, 0 destroyed.
user@debian:~/tform$ terraform destroy
macaddress.vmmac: Refreshing state... [id=16:43:7e:85:3b:8c]
opnsense_kea_dhcpv4_reservation.debian_clone: Refreshing state... [id=9d6ad7c3-0e26-4b9f-8e56-ad5271bc2399]
proxmox_vm_qemu.debian_clone: Refreshing state... [id=pve/qemu/120]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the
following symbols:
- destroy

Terraform will perform the following actions:

# macaddress.vmmac will be destroyed
- resource "macaddress" "vmmac" {
  - address = "16:43:7e:85:3b:8c" -> null
  - id      = "16:43:7e:85:3b:8c" -> null
}

# opnsense_kea_dhcpv4_reservation.debian_clone will be destroyed
- resource "opnsense_kea_dhcpv4_reservation" "debian_clone" {
  - description = "Static IP for tf_cloned_debian" -> null
  - hostname   = "tf_cloned_debian" -> null
  - id         = "9d6ad7c3-0e26-4b9f-8e56-ad5271bc2399" -> null
  - ip_address = "10.0.0.140" -> null
  - mac_address = "16:43:7e:85:3b:8c" -> null
  - subnet_id  = "4f275fc3-7f2d-4bdb-b8a7-eb794075aa7f" -> null
}

# proxmox_vm_qemu.debian_clone will be destroyed

Plan: 0 to add, 0 to change, 3 to destroy.

Do you really want to destroy all resources?
  Terraform will destroy all your managed infrastructure, as shown above.
  There is no undo. Only 'yes' will be accepted to confirm.

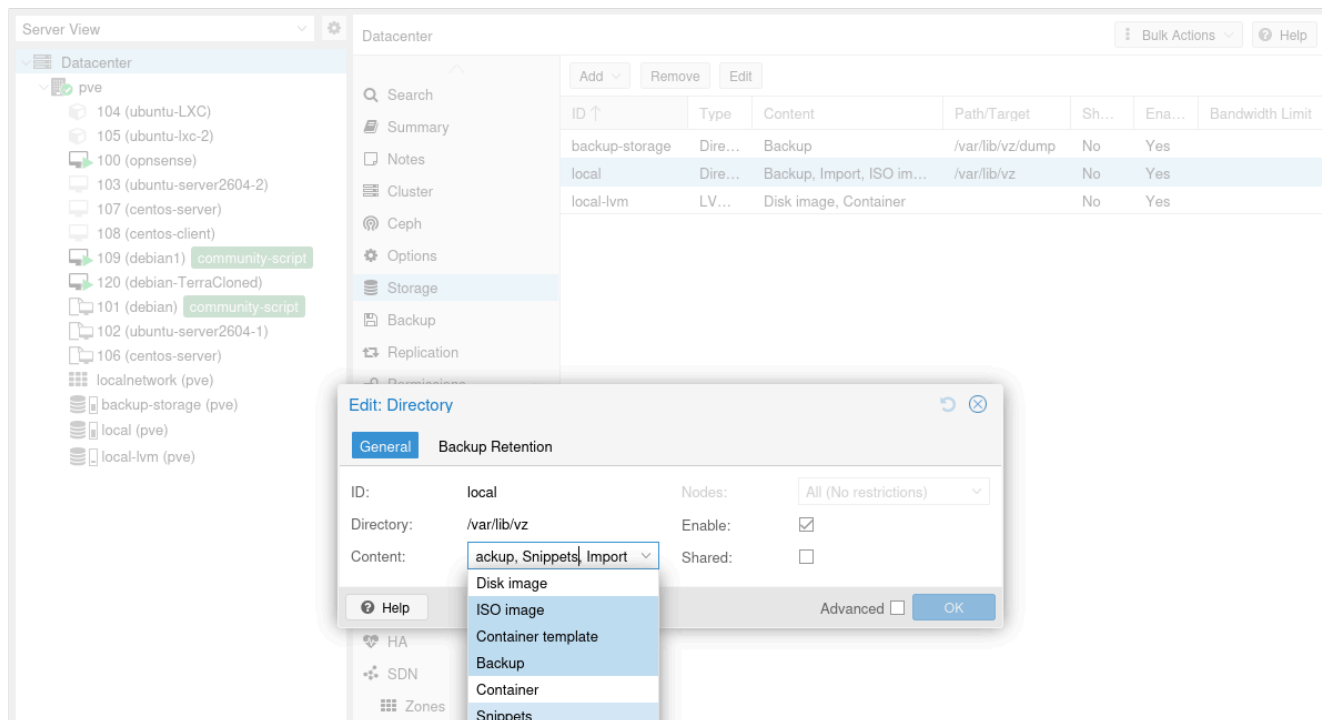
Enter a value: yes

opnsense_kea_dhcpv4_reservation.debian_clone: Destroying... [id=9d6ad7c3-0e26-4b9f-8e56-ad5271bc2399]
proxmox_vm_qemu.debian_clone: Destroying... [id=pve/qemu/120]
opnsense_kea_dhcpv4_reservation.debian_clone: Destruction complete after 1s
proxmox_vm_qemu.debian_clone: Destruction complete after 4s
macaddress.vmmac: Destroying... [id=16:43:7e:85:3b:8c]
macaddress.vmmac: Destruction complete after 0s

Destroy complete! Resources: 3 destroyed.
user@debian:~/tform$
```

3. Implémentation de Cloud-init

3.1. Activation snippets dans local



3.2. Création d'un yaml pour Cloud-init

```
1 #cloud-config
2 hostname: sti-web01
3
4 users:
5   - default
6   - name: ${ci_user}
7     groups: [sudo]
8     shell: /bin/bash
9     sudo: ["ALL=(ALL) NOPASSWD:ALL"]
10    ssh_authorized_keys:
11      - ${ssh_public_key}
12
13 package_update: true
14 package_upgrade: true
15 packages:
16   - vim
17   - curl
18   - nginx
19
20 runcmd:
21   - systemctl enable qemu-guest-agent
22   - systemctl start qemu-guest-agent
```

Les package installés sont vim, curl et nginx.

3.3. Gestion de l'envoi du yml dans local:snippets

```
main.tf
1  resource "terraform_data" "upload_snippet" {
2      triggers_replace = {
3          snippet_hash = filemd5("user-data.yml")
4      }
5
6      connection {
7          type      = "ssh"
8          user      = "root"
9          private_key = file(pathexpand("~/ssh/id_ed25519"))
10         host      = "192.168.122.220"
11     }
12
13     provisioner "remote-exec" {
14         inline = [
15             "mkdir -p /var/lib/vz/snippets/"
16         ]
17     }
18
19     provisioner "file" {
20         source      = "user-data.yml"
21         destination = "/var/lib/vz/snippets/user-data.yml"
22     }
23 }
```

Il faut donc:

- Vérifier (avec le hash) si le fichier de config est différent de celui présent sur local:snippets
- Ouvre une session SSH root
- Crée le dossier /var/lib/vz/snippets/ dans le disque local (-p pour ne pas renvoyer d'erreur s'il existe déjà)
- Effectue la copie via scp

3.4. Dernières modifications du main.tf

main.tf

```
1 resource "proxmox_vm_qemu" "debian_clone" {
2   name          = "debian-TerraCloned"
3   target_node   = "pve"
4
5   depends_on    = [terraform_data.upload_snippet] #ajout de la dépendance
6
7   os_type       = "cloud-init" #modif ici
8   ciuser        = var.ci_user #les variables ciuser et ssh_public_key_path sont stockées dans un
                        #var.tf associé, qui servira a stocker l'ensembles des variables pour + de propreté
9   sshkeys       = trimspace(file(pathexpand(var.ssh_public_key_path)))
10  clone         = "debian"
11  full_clone    = true
12  agent         = 1
13  start_at_node_boot = true
14  # onboot = true
15  ipconfig0     = "ip=dhcp"
16  cicustom      = "user=local:snippets/user-data.yml" #ajout ici
```

3.5. Terraform plan

```
user@debian:~/tform$ terraform plan
```

Terraform used the selected providers to generate the following execution plan. F
actions are indicated with the following symbols:

+ create

Terraform will perform the following actions:

```
# macaddress.vmmac will be created
+ resource "macaddress" "vmmac" {
  + address = (known after apply)
  + id      = (known after apply)
}

# opnsense_kea_dhcpv4_reservation.debian_clone will be created
+ resource "opnsense_kea_dhcpv4_reservation" "debian_clone" {
  + description = "Static IP for tf_cloned_debian"
  + hostname    = "tf_cloned_debian"
  + id          = (known after apply)
  + ip_address  = "10.0.0.140"
  + mac_address = (known after apply)
  + subnet_id   = "4f275fc3-7f2d-4bdb-b8a7-eb794075aa7f"
}

# proxmox_vm_qemu.debian_clone will be created
+ resource "proxmox_vm_qemu" "debian_clone" {
  + additional_wait      = 5
  + agent                = 1
  + agent_timeout        = 90
  + automatic_reboot     = true
  + automatic_reboot_severity = "error"
  + balloon              = 0
  + bios                 = "seabios"
  + boot                 = (known after apply)
  + bootdisk             = (known after apply)
  + cicustom             = "user=local:snippets/user-data.yml"
  + ciupgrade            = false
  + ciuser               = "admin"
  + clone                = "debian"
  + clone_wait           = 10
  + current_node         = (known after apply)
  + default_ipv4_address = (known after apply)
  + default_ipv6_address = (known after apply)
  + define_connection_info = true
  + description          = "Managed by Terraform."
  + force_create         = false
  + full_clone           = true
  + hotplug              = "network,disk,usb"
  + id                   = (known after apply)
  + ipconfig0            = "ip=dhcp"
  + kvm                  = true
}
```

```
+ linked_vmid          = (known after apply)
+ memory              = 2048
+ name                = "debian-TerraCloned"
+ os_type             = "cloud-init"
+ protection          = false
+ reboot_required     = (known after apply)
+ scsihw              = "lsi"
+ skip_ipv4           = false
+ skip_ipv6           = true
+ ssh_host            = (known after apply)
+ ssh_port            = (known after apply)
+ sshkeys             = "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIDcr42Cr
JQHGM4c0CQPjYyT0W3W74RtL0ydw user@debian"
+ start_at_node_boot  = true
+ tablet              = true
+ target_node         = "pve"
+ unused_disk         = (known after apply)
+ vm_state            = "running"
+ vmid                = 120

+ cpu {
  + cores    = 2
  + limit    = 0
  + numa     = false
  + sockets  = 1
  + type     = "host"
  + units    = 0
  + vcores   = 0
}

+ disk {
  + backup          = true
  + id              = (known after apply)
  + iops_r_burst    = 0
  + iops_r_burst_length = 0
  + iops_r_concurrent = 0
  + iops_wr_burst   = 0
  + iops_wr_burst_length = 0
  + iops_wr_concurrent = 0
  + linked_disk_id  = (known after apply)
  + mbytes_r_burst  = 0
  + mbytes_r_concurrent = 0
  + mbytes_wr_burst = 0
  + mbytes_wr_concurrent = 0
  + passthrough     = false
  + size            = "40G"
  + slot            = "scsi0"
  + storage         = "local-lvm"
```

```
    + type = "disk"
  }

+ network {
  + bridge = "vmbri1"
  + firewall = false
  + id = 0
  + link_down = false
  + macaddr = (known after apply)
  + model = "virtio"
}

+ smbios (known after apply)
}

# terraform_data.upload_snippet will be created
+ resource "terraform_data" "upload_snippet" {
  + id = (known after apply)
  + triggers_replace = {
    + snippet_hash = "76630081873a374637e8471c617d469e"
  }
}
```

Plan: 4 to add, 0 to change, 0 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee exactly these actions if you run "terraform apply" now.

user@debian:~/tform\$

3.6. Terraform apply

```
user@debian:~/tform$ terraform apply
```

Terraform used the selected providers to generate the following execution plan. For actions indicated with the following symbols:

- + create

Terraform will perform the following actions:

```
# macaddress.vmmac will be created
+ resource "macaddress" "vmmac" {
  + address = (known after apply)
  + id      = (known after apply)
}

# opnsense_kea_dhcpv4_reservation.debian_clone will be created
+ resource "opnsense_kea_dhcpv4_reservation" "debian_clone" {
  + description = "Static IP for tf_cloned_debian"
  + hostname    = "tf_cloned_debian"
  + id          = (known after apply)
  + ip_address  = "10.0.0.140"
  + mac_address = (known after apply)
  + subnet_id   = "4f275fc3-7f2d-4bdb-b8a7-eb794075aa7f"
}

# proxmox_vm_qemu.debian_clone will be created
+ resource "proxmox_vm_qemu" "debian_clone" {
  + additional_wait = 5
  + agent           = 1
  + agent_timeout   = 90
  + automatic_reboot = true
  + automatic_reboot_severity = "error"
  + balloon         = 0
  + bios            = "seabios"
  + boot            = (known after apply)
  + bootdisk        = (known after apply)
  + cicustom        = "user=local:snippets/user-data.yml"
  + ciupgrade       = false
  + ciuser          = "admin"
  + clone           = "debian"
  + clone_wait      = 10
  + current_node    = (known after apply)
  + default_ipv4_address = (known after apply)
  + default_ipv6_address = (known after apply)
  + define_connection_info = true
  + description     = "Managed by Terraform."
  + force_create    = false
  + full_clone      = true
  + hotplug         = "network,disk,usb"
  + id              = (known after apply)
  + ipconfig0       = "ip=dhcp"
```

```
+ kvm = true
+ linked_vmid = (known after apply)
+ memory = 2048
+ name = "debian-TerraCloned"
+ os_type = "cloud-init"
+ protection = false
+ reboot_required = (known after apply)
+ scsihw = "lsi"
+ skip_ipv4 = false
+ skip_ipv6 = true
+ ssh_host = (known after apply)
+ ssh_port = (known after apply)
+ sshkeys = "ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIDcr42Cr
JQHGM4c0CQPjYyT0W3W74RtL0ydw user@debian"
+ start_at_node_boot = true
+ tablet = true
+ target_node = "pve"
+ unused_disk = (known after apply)
+ vm_state = "running"
+ vmid = 120

+ cpu {
  + cores = 2
  + limit = 0
  + numa = false
  + sockets = 1
  + type = "host"
  + units = 0
  + vcores = 0
}

+ disk {
  + backup = true
  + id = (known after apply)
  + iops_r_burst = 0
  + iops_r_burst_length = 0
  + iops_r_concurrent = 0
  + iops_wr_burst = 0
  + iops_wr_burst_length = 0
  + iops_wr_concurrent = 0
  + linked_disk_id = (known after apply)
  + mbps_r_burst = 0
  + mbps_r_concurrent = 0
  + mbps_wr_burst = 0
  + mbps_wr_concurrent = 0
  + passthrough = false
  + size = "40G"
  + slot = "scsi0"
```

```
+ storage          = "local-lvm"
+ type             = "disk"
}

+ network {
+   bridge      = "vmbri1"
+   firewall    = false
+   id          = 0
+   link_down   = false
+   macaddr     = (known after apply)
+   model       = "virtio"
}

+ smbios (known after apply)
}

# terraform_data.upload_snippet will be created
+ resource "terraform_data" "upload_snippet" {
+   id          = (known after apply)
+   triggers_replace = {
+     snippet_hash = "76630081873a374637e8471c617d469e"
+   }
}
```

Plan: 4 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?

Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

```
terraform_data.upload_snippet: Creating...
terraform_data.upload_snippet: Provisioning with 'remote-exec'...
terraform_data.upload_snippet (remote-exec): Connecting to remote host via SSH...
terraform_data.upload_snippet (remote-exec):   Host: 192.168.122.220
terraform_data.upload_snippet (remote-exec):   User: root
terraform_data.upload_snippet (remote-exec):   Password: false
terraform_data.upload_snippet (remote-exec):   Private key: true
terraform_data.upload_snippet (remote-exec):   Certificate: false
terraform_data.upload_snippet (remote-exec):   SSH Agent: true
terraform_data.upload_snippet (remote-exec):   Checking Host Key: false
terraform_data.upload_snippet (remote-exec):   Target Platform: unix
macaddress.vmmac: Creating...
macaddress.vmmac: Creation complete after 0s [id=e6:a6:fb:1c:85:04]
opnsense_kea_dhcpv4_reservation.debian_clone: Creating...
opnsense_kea_dhcpv4_reservation.debian_clone: Creation complete after 1s [id=926c
f-4996-b29e-21d7df1c5490]
```

```
terraform_data.upload_snippet (remote-exec): Connected!
terraform_data.upload_snippet: Provisioning with 'file'...
terraform_data.upload_snippet: Creation complete after 1s [id=61d91797-848e-6914-401291358]
proxmox_vm_qemu.debian_clone: Creating...
proxmox_vm_qemu.debian_clone: Still creating... [00m10s elapsed]
proxmox_vm_qemu.debian_clone: Still creating... [00m20s elapsed]
proxmox_vm_qemu.debian_clone: Still creating... [00m30s elapsed]
proxmox_vm_qemu.debian_clone: Still creating... [00m40s elapsed]
proxmox_vm_qemu.debian_clone: Still creating... [00m50s elapsed]
proxmox_vm_qemu.debian_clone: Creation complete after 55s [id=pve/qemu/120]

Apply complete! Resources: 4 added, 0 changed, 0 destroyed.
user@debian:~/tform$
```

vérification:

```
root@localhost:~# curl 127.0.0.1 > output.txt && head -n 5 output.txt
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           %             0         1639k    0  -----  -----  ----- 1741k
<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN" "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml">
  <head>
    <meta http-equiv="Content-Type" content="text/html; charset=UTF-8" />
root@localhost:~#
```

cURL fonctionne parfaitement

```
root@localhost:~# vim output.txt > tqf.txt
Vim: Warning: Output is not to a terminal
```

Vim aussi

```
root@localhost:~# sudo apt update && apt upgrade
Get:1 file:/etc/apt/mirrors/debian.list Mirrorlist [30 B]
Get:2 file:/etc/apt/mirrors/debian-security.list Mirrorlist [39 B]
Hit:3 https://deb.debian.org/debian bookworm InRelease
Get:4 https://deb.debian.org/debian bookworm-updates InRelease [55.4 kB]
Get:5 https://deb.debian.org/debian bookworm-backports InRelease [59.4 kB]
Get:6 https://deb.debian.org/debian-security bookworm-security InRelease [34.8 kB]
Get:7 https://deb.debian.org/debian bookworm-backports/main Sources.diff/Index [63.3 kB]
Get:11 https://deb.debian.org/debian bookworm-backports/main Sources T-2026-09-14-1401.50-F-2026-09-14-1401.50.pdiff [299 B]
Get:11 https://deb.debian.org/debian bookworm-backports/main Sources T-2026-09-14-1401.50-F-2026-09-14-1401.50.pdiff [299 B]
Get:8 https://deb.debian.org/debian-security bookworm-security/main Sources [248 kB]
Get:9 https://deb.debian.org/debian-security bookworm-security/main amd64 Packages [338 kB]
Get:10 https://deb.debian.org/debian-security bookworm-security/main Translation-en [211 kB]
Fetched 1010 kB in 1s (870 kB/s)
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
All packages are up to date.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
0 upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
root@localhost:~#
```

Les paquets sont à jour

3.7. Terraform destroy

```
user@debian:~/tform$ terraform destroy
terraform_data.upload_snippet: Refreshing state... [id=61d91797-848e-6914-2373-fe58]
macaddress.vmmac: Refreshing state... [id=e6:a6:fb:1c:85:04]
opnsense_kea_dhcpv4_reservation.debian_clone: Refreshing state... [id=926d685a-f8b29e-21d7df1c5490]
proxmox_vm_qemu.debian_clone: Refreshing state... [id=pve/qemu/120]
```

Terraform used the selected providers to generate the following execution plan. For actions are indicated with the following symbols:

- destroy

Terraform will perform the following actions:

```
# macaddress.vmmac will be destroyed
- resource "macaddress" "vmmac" {
  - address = "e6:a6:fb:1c:85:04" -> null
  - id      = "e6:a6:fb:1c:85:04" -> null
}

# opnsense_kea_dhcpv4_reservation.debian_clone will be destroyed
- resource "opnsense_kea_dhcpv4_reservation" "debian_clone" {
  - description = "Static IP for tf_cloned_debian" -> null
  - hostname    = "tf_cloned_debian" -> null
  - id          = "926d685a-f8ef-4996-b29e-21d7df1c5490" -> null
  - ip_address  = "10.0.0.140" -> null
  - mac_address = "e6:a6:fb:1c:85:04" -> null
  - subnet_id   = "4f275fc3-7f2d-4bdb-b8a7-eb794075aa7f" -> null
}

# proxmox_vm_qemu.debian_clone will be destroyed
- resource "proxmox_vm_qemu" "debian_clone" {
  - additional_wait      = 5 -> null
  - agent                = 1 -> null
  - agent_timeout        = 90 -> null
  - automatic_reboot     = true -> null
  - automatic_reboot_severity = "error" -> null
  - balloon              = 0 -> null
  - bios                 = "seabios" -> null
  - boot                 = "order=scsi0" -> null
  - cicustom             = "user=local:snippets/user-data.yml" -> null
  - ciupgrade            = false -> null
  - ciuser               = "admin" -> null
  - clone                = "debian" -> null
  - clone_wait           = 10 -> null
  - cores                = 0 -> null
  - current_node         = "pve" -> null
  - default_ipv4_address = "10.0.0.140" -> null
  - define_connection_info = true -> null
  - description          = "Managed by Terraform." -> null
}
```

```
- force_create          = false -> null
- full_clone           = true -> null
- hotplug              = "network,disk,usb" -> null
- id                   = "pve/qemu/120" -> null
- ipconfig0            = "ip=dhcp" -> null
- kvm                  = true -> null
- linked_vmid          = 0 -> null
- memory               = 2048 -> null
- name                 = "debian-TerraCloned" -> null
- numa                 = false -> null
- os_type               = "cloud-init" -> null
- protection           = false -> null
- qemu_os              = "l26" -> null
- reboot_required      = false -> null
- scsihw               = "lsi" -> null
- skip_ipv4             = false -> null
- skip_ipv6            = true -> null
- sockets              = 0 -> null
- ssh_host             = "10.0.0.140" -> null
- ssh_port             = "22" -> null
- sshkeys              = <<-EOT
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIDcr42CrxxnGHxyUJQHGM4c0CQPjYyT0V
ydw user@debian
EOT -> null
- start_at_node_boot   = true -> null
- tablet               = true -> null
- tags                 = null
- target_node          = "pve" -> null
- unused_disk          = [] -> null
- vcpus                = 0 -> null
- vm_state             = "running" -> null
- vmid                 = 120 -> null
# (9 unchanged attributes hidden)

- cpu {
  - cores      = 2 -> null
  - limit      = 0 -> null
  - numa       = false -> null
  - sockets    = 1 -> null
  - type       = "host" -> null
  - units      = 0 -> null
  - vcores     = 0 -> null
  # (1 unchanged attribute hidden)
}

- disk {
  - backup      = true -> null
  - discard     = false -> null
```

```
- emulatessd          = false -> null
- format              = "raw" -> null
- id                  = 1 -> null
- iops_r_burst        = 0 -> null
- iops_r_burst_length = 0 -> null
- iops_r_concurrent   = 0 -> null
- iops_wr_burst       = 0 -> null
- iops_wr_burst_length = 0 -> null
- iops_wr_concurrent  = 0 -> null
- iothread            = false -> null
- linked_disk_id      = -1 -> null
- mbps_r_burst        = 0 -> null
- mbps_r_concurrent   = 0 -> null
- mbps_wr_burst       = 0 -> null
- mbps_wr_concurrent  = 0 -> null
- passthrough         = false -> null
- readonly            = false -> null
- replicate           = false -> null
- size                = "40G" -> null
- slot               = "scsi0" -> null
- storage             = "local-lvm" -> null
- type                = "disk" -> null
  # (6 unchanged attributes hidden)
}

- network {
  - bridge    = "vmb1" -> null
  - firewall  = false -> null
  - id        = 0 -> null
  - link_down = false -> null
  - macaddr   = "E6:A6:FB:1C:85:04" -> null
  - model     = "virtio" -> null
  - mtu       = 0 -> null
  - queues    = 0 -> null
  - rate      = 0 -> null
  - tag       = 0 -> null
}

- smbios {
  - uuid      = "58d80270-a06a-4187-9072-dc8e273c92fd" -> null
  # (6 unchanged attributes hidden)
}

- startup_shutdown {
  - order          = -1 -> null
  - shutdown_timeout = -1 -> null
  - startup_delay   = -1 -> null
}
```

```
}  
}
```

```
# terraform_data.upload_snippet will be destroyed  
- resource "terraform_data" "upload_snippet" {  
  - id = "61d91797-848e-6914-2373-feb401291358" -> null  
  - triggers_replace = {  
    - snippet_hash = "76630081873a374637e8471c617d469e"  
  } -> null  
}
```

Plan: 0 to add, 0 to change, 4 to destroy.

Do you really want to destroy all resources?

Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

opnsense_kea_dhcpv4_reservation.debian_clone: Destroying... [id=926d685a-f8ef-4991d7df1c5490]

proxmox_vm_qemu.debian_clone: Destroying... [id=pve/qemu/120]

opnsense_kea_dhcpv4_reservation.debian_clone: Destruction complete after 1s

proxmox_vm_qemu.debian_clone: Destruction complete after 4s

terraform_data.upload_snippet: Destroying... [id=61d91797-848e-6914-2373-feb401291358]

macaddress.vmmac: Destroying... [id=e6:a6:fb:1c:85:04]

terraform_data.upload_snippet: Destruction complete after 0s

macaddress.vmmac: Destruction complete after 0s

Destroy complete! Resources: 4 destroyed.

user@debian:~/tform\$ █

4. Cleanup de l'espace de travail

Pour éviter de commiter des token, ranger correctement les variables et les providers, notamment pour travailler avec le versionnage git, il convient de ranger ces informations dans différents fichiers:

```
1 user@debian:~/terraform$ ls
2 main.tf provider.tf secrets.auto.tfvars terraform.tfstate user-data.yml.tpl var.tf
3 user@debian:~/terraform$ cat .gitignore
4 *.tfvars
5 *.tfvars.json
6
7 **/.terraform/*
8 *.tfstate
9 *.tfstate.*
10 crash.log
11 crash.*.log
12 *.tfplan
```

```
1 user@debian:~/terraform$ cat provider.tf
2 terraform {
3   required_providers {
4     proxmox = {
5       source = "telmate/proxmox"
6       version = "3.0.2-rc07" #3.0.2-rc07
7     }
8     macaddress = {
9       source = "ivoronin/macaddress"
10      version = "0.3.0"
11    }
12    opnsense = {
13      source = "browningluke/opnsense"
14      version = "~> 0.26.0"
15    }
16  }
17 }
18
19 provider "proxmox" {
20   pm_api_url          = var.pm_api_url
21   pm_api_token_id     = var.pm_api_token_id
22   pm_api_token_secret = var.pm_api_token_secret
23   pm_tls_insecure     = true
24   pm_minimum_permission_check = false
25 }
26
27 provider "opnsense" {
28   uri          = var.opnsense_uri
29   api_key      = var.opnsense_api_key
30   api_secret   = var.opnsense_api_secret
31   allow_insecure = true
32 }
```

```
1 user@debian:~/terraform$ cat secrets.auto.tfvars
2 pm_api_url           = "https://192.168.122.220:8006/api2/json"
3 pm_api_token_id      = "terraform@pve!tf-token"
4 pm_api_token_secret  = "2ef5d121-2f70-48c7-a17b-78e7dc22d76e"
5
6 opnsense_uri         = "https://192.168.122.197"
7 opnsense_api_key     = "AncmTmUYXl5csabprgylFjDahbfRohVt7jYNm+UbKnPwAApQXaHXV7K3/
  UeYLUkcFYlnCsRXYtnua/DC"
8 opnsense_api_secret  =
  "JU4/5kxzlFm5tALdiscA5Bzt1s6F+TZh6GoY2JLf0ITYUVadoVs4CjEq8lGH2kV1SNLaRLKQvVKxd370"
```

```
1 user@debian:~/terraform$ cat main.tf
2 locals {
3   ssh_public_key = trimspace(file(pathexpand(var.ssh_public_key_path)))
4 }
5
6 resource "terraform_data" "upload_snippet" {
7   triggers_replace = {
8     snippet_hash = md5(templatefile("${path.module}/user-data.yml.tpl", {
9       ci_user      = var.ci_user
10      ssh_public_key = local.ssh_public_key
11    }))
12   }
13
14   connection {
15     type      = "ssh"
16     user      = "root"
17     private_key = file(pathexpand("~/ssh/id_ed25519"))
18     host      = "192.168.122.220"
19   }
20
21   provisioner "remote-exec" {
22     inline = [
23       "mkdir -p /var/lib/vz/snippets/"
24     ]
25   }
26
27   provisioner "file" {
28     content = templatefile("${path.module}/user-data.yml.tpl", {
29       ci_user      = var.ci_user
30       ssh_public_key = local.ssh_public_key
31     })
32     destination = "/var/lib/vz/snippets/user-data.yml"
33   }
34 }
35
36 resource "macaddress" "vmmac" {
37   # prefix = [52, 54, 00]
38 }
39
```

```
40 resource "opnsense_kea_dhcpv4_reservation" "debian_clone" {
41   subnet_id   = "4f275fc3-7f2d-4bdb-b8a7-eb794075aa7f"
42   ip_address  = "10.0.0.140"
43   mac_address = lower(macaddress.vmmac.address)
44   hostname    = "tf_cloned_debian"
45   description = "Static IP for tf_cloned_debian"
46 }
47
48 resource "proxmox_vm_qemu" "debian_clone" {
49   name          = "debian-TerraCloned"
50   target_node   = "pve"
51
52   depends_on    = [terraform_data.upload_snippet]
53
54   os_type       = "cloud-init"
55   ciuser        = var.ci_user
56   sshkeys       = local.ssh_public_key
57   clone         = "debian"
58   full_clone    = true
59   agent         = 1
60   start_at_node_boot = true
61   # onboot = true
62   ipconfig0     = "ip=dhcp"
63   cicustom      = "user=local:snippets/user-data.yml"
64
65   vmid          = 120
66   memory        = 2048
67   skip_ipv6     = true
68
69   cpu {
70     cores    = 2
71     sockets  = 1
72     type     = "host"
73   }
74
75   disk {
76     slot    = "scsi0" #ide2
77     size    = "40G"
78     type    = "disk" # cloudinit
79     storage = "local-lvm"
80   }
81
82   network {
83     id      = 0
84     model   = "virtio"
85     bridge  = "vmbr1"
86     macaddr = upper(macaddress.vmmac.address)
87   }
88 }
```

```
1 user@debian:~/terraform$ cat var.tf
2 variable "ci_user" {
3     description = "Utilisateur créé via cloud-init sur la VM"
4     type        = string
5     default     = "admin"
6 }
7
8 variable "ssh_public_key_path" {
9     description = "Chemin local vers la clé publique SSH à injecter dans la VM"
10    type        = string
11    default     = "~/ssh/id_ed25519.pub"
12 }
13
14 variable "pm_api_url" {
15     description = "URL de l'API Proxmox (ex: https://host:8006/api2/json)"
16     type        = string
17 }
18
19 variable "pm_api_token_id" {
20     description = "Identifiant du token API Proxmox (ex: terraform@pve!tf-token)"
21     type        = string
22 }
23
24 variable "pm_api_token_secret" {
25     description = "Secret du token API Proxmox"
26     type        = string
27     sensitive   = true
28 }
29
30 variable "opnsense_uri" {
31     description = "URL de l'API OPNsense"
32     type        = string
33 }
34
35 variable "opnsense_api_key" {
36     description = "Clé API OPNsense"
37     type        = string
38     sensitive   = true
39 }
40
41 variable "opnsense_api_secret" {
42     description = "Secret API OPNsense"
43     type        = string
44     sensitive   = true
45 }
```

5. Exercice final : une plateforme WordPress

Cette seconde partie met en œuvre l'exercice final du cours : une plateforme WordPress composée d'un répartiteur HAProxy, de deux nœuds nginx + WordPress et d'une base MariaDB partagée. Comme dans le cours, le travail est partagé entre trois outils : Terraform crée les VM, cloud-init les prépare au premier démarrage, Ansible installe et configure les logiciels.

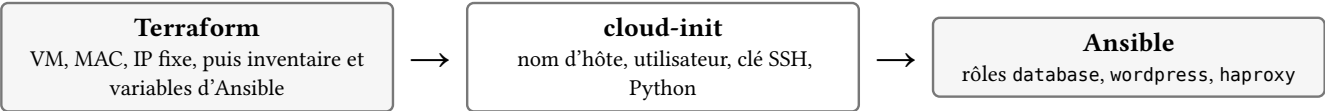


Schéma 1 : Chaîne de déploiement

cloud-init ne s'exécute qu'une fois, au premier démarrage d'une VM. Ansible peut au contraire être relancé à volonté : il compare l'état de chaque VM à celui que décrivent ses rôles et ne modifie que ce qui diffère. C'est ce qui rend la plateforme rejouable, un des critères du cours.

Toutes les VM sont clonées depuis le même modèle debian-cloudinit et décrites par une seule table Terraform, `var.vms` (nom, rôle, vmid, IP fixe). Chacune garde la structure de la première partie ce projet: adresse MAC générée, associée à une IP fixe par réservation dans le Kea d'OpenSense, snippet cloud-init. Elles ont été ajoutées une à une, en vérifiant à chaque étape que terraform plan ne détruit rien et qu'un second passage d'Ansible ne change rien (voir git log).

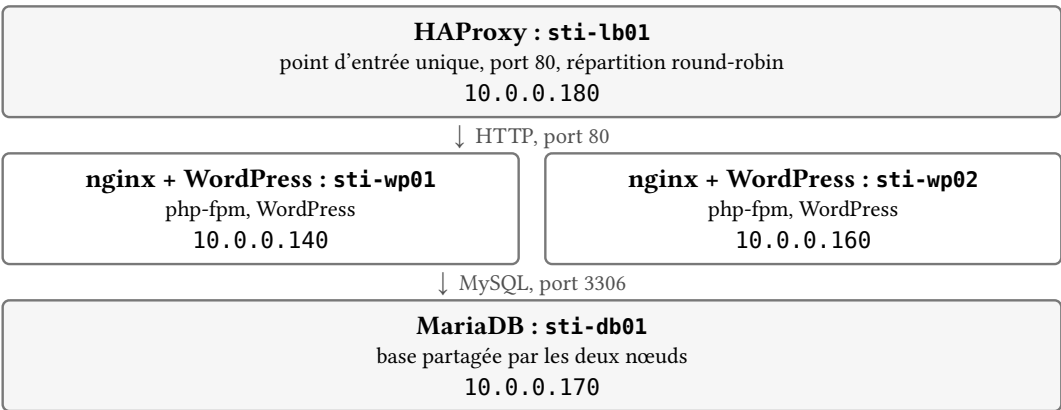


Schéma 2 : Architecture cible de la plateforme

VM	Rôle	vmid	IP fixe	Installé par Ansible
sti-wp01	wordpress	120	10.0.0.140	nginx, php-fpm et extensions PHP, WordPress
sti-wp02	wordpress	121	10.0.0.160	identique à sti-wp01
sti-db01	database	122	10.0.0.170	MariaDB, base et compte de WordPress
sti-lb01	haproxy	123	10.0.0.180	HAProxy, rsyslog

Le déploiement complet tient en trois commandes : `terraform apply` (VM, puis inventaire et variables d'Ansible) puis, depuis `ansible/`, `ansible-galaxy collection install -r requirements.yml` (une fois : modules MariaDB) et `ansible-playbook site.yml`. Les extraits de code qui suivent viennent des fichiers finaux du projet ; « [...] » marquant des lignes omises.

Fichier	Rôle
<code>var.tf</code> , <code>main.tf</code> , <code>provider.tf</code>	Terraform : variables, ressources (VM, MAC, Kea, snippets, règle OPNsense), inventaire et variables d'Ansible
<code>user-data.yml.tpl</code> , <code>inventory.tpl</code>	cloud-init de base, commun à toutes les VM ; modèle de l'inventaire
<code>ansible/</code>	<code>site.yml</code> (playbook), <code>roles/<rôle>/</code> (un rôle par type de VM), <code>inventory.ini</code> et <code>group_vars/</code> (générés par Terraform, jamais commités)

5.1. Implémentation de WordPress avec nginx

La première VM de la plateforme, `sti-wp01` (vmid 120, 10.0.0.140), est la VM de la première partie transformée en nœud WordPress. Son adresse MAC et son IP fixe ne changent pas. Son cloud-init se limite désormais à la préparation de base ; nginx, php-fpm et WordPress sont installés par le rôle Ansible `wordpress`.

5.1.1. Déroulement

1. Le cloud-init de la première partie ne garde que la préparation de base : vim et curl comme avant (nginx n'y est plus installé), un utilisateur admin avec sa clé SSH et python3 avec python3-apt (pour le module apt), que suppose Ansible. Il est identique pour toutes les VM de la plateforme.
2. Une fois la VM créée, Terraform écrit trois fichiers dans `ansible/` : l'inventaire (un groupe par rôle de la table `vms`, avec l'IP fixe de chaque VM), les variables communes et, à part, le mot de passe de la base. Les valeurs ne sont saisies qu'à un seul endroit : les variables Terraform.
3. Le rôle `wordpress` décrit l'état voulu du nœud : nginx, php-fpm et ses extensions installés, WordPress téléchargé une seule fois dans la version fixée par `wp_version` (empreinte SHA1 vérifiée), `wp-config.php` généré depuis un modèle, site nginx actif.
4. Aucune base n'est installée sur la VM : `wp-config.php` désigne MariaDB (`sti-db01`), déployée à l'étape suivante. D'ici là, la sonde `/readyz` répond 503 (base injoignable) et le playbook l'accepte.
5. Chaque fichier de configuration est contrôlé avant d'être installé (`php -l` pour `wp-config.php`, `nginx -t` pour nginx), et nginx n'est rechargé que si sa configuration a changé.
6. `ansible-playbook site.yml` attend d'abord que la VM réponde en SSH et que cloud-init ait terminé, puis applique le rôle. Rejoué, il ne change rien (`changed=0`).

5.1.2. Ajouts dans les fichiers Terraform

Dans `provider.tf`, le `provider local` permet à Terraform d'écrire des fichiers sur le poste ; `terraform init` doit être relancé pour le télécharger.

`provider.tf`

```
1 local = {  
2     source = "hashicorp/local"  
3     version = "~> 2.5"  
4 }
```

Dans `var.tf`, trois variables : la version de WordPress (`wp_version`, 7.1.2 par défaut), le mot de passe de la base et, facultatif, un hôte de rebond SSH par lequel Ansible joint les VM si elles ne sont pas routées depuis le poste (`null` : connexion directe). Le mot de passe est `sensitive`, fait au moins 12 caractères et n'utilise que des caractères sûrs, car il est inséré dans du PHP et du SQL ; il est défini dans un fichier `.tfvars` ignoré par git (chapitre 4).

`var.tf`

```
1 variable "wp_db_password" {  
2     [...]  
3     sensitive = true  
4     [...]  
5 }  
6  
7 variable "ansible_jump_host" {  
8     [...]  
9     default = null  
10    [...]  
11 }
```

Dans `main.tf`, deux valeurs calculées (`locals`) préparent le passage de relais : les groupes de l'inventaire (un par rôle, avec le nom et l'IP de chaque VM de ce rôle, tirés de la table `vms`) et les variables partagées avec Ansible. Côté Ansible, les variables d'un rôle portent le nom du rôle ; `wp_db_name` et `wp_db_user` sont communes à WordPress et à la base. L'hôte de la base, `wordpress_db_host`, est calculé aussi : il vaut `sti-db01`, le nom prévu, tant que cette VM n'existe pas dans la table (voir la sous-partie MariaDB).

```
main.tf
1  locals {
2    ssh_private_key_path = trimsuffix(var.ssh_public_key_path, ".pub")
3    [...]
4    ansible_groups = [
5      for role in ["database", "wordpress", "haproxy"] : {
6        role = role
7        hosts = { for name, vm in var.vms : name => vm.ip if vm.role == role }
8      }
9    ]
10   [...]
11   ansible_vars = {
12     wp_db_name      = var.wp_db_name
13     wp_db_user      = var.wp_db_user
14     wordpress_version = var.wp_version
15     [...]
16   }
17 }
```

L'inventaire est écrit par une ressource du provider `local`, une fois les VM créées (`depends_on`), à partir du modèle `inventory.tpl`.

```
main.tf
1  resource "local_file" "ansible_inventory" {
2    filename      = "${path.module}/ansible/inventory.ini"
3    [...]
4    content = templatefile("${path.module}/inventory.tpl", {
5      groups      = local ansible_groups
6      ci_user     = var.ci_user
7      private_key = local.ssh_private_key_path
8      jump_host   = var.ansible_jump_host
9    })
10
11   depends_on = [proxmox_vm_qemu.vm]
12 }
```

Les variables communes vont dans `group_vars/all/terraform.yml`. Le mot de passe est écrit à part (`local_sensitive_file`, droits `0600`) : son contenu n'apparaît jamais dans la sortie de `terraform plan`.

`main.tf`

```
1  resource "local_file" "ansible_vars" {
2    filename          = "${path.module}/ansible/group_vars/all/terraform.yml"
3    [...]
4    yamlencode(local.ansible_vars),
5    [...]
6  }
7
8  resource "local_sensitive_file" "ansible_secrets" {
9    filename          = "${path.module}/ansible/group_vars/all/secrets.yml"
10   file_permission    = "0600"
11   [...]
12   yamlencode({ wp_db_password = var.wp_db_password }),
13   [...]
14 }
```

`inventory.tpl` écrit un groupe par rôle, avec une ligne par VM (nom `ansible_host=IP`), puis les paramètres de connexion communs : utilisateur créé par `cloud-init`, clé SSH privée, interpréteur Python et, si `ansible_jump_host` est renseigné, le rebond SSH. Un groupe sans VM reste présent, vide : le playbook est le même à chaque étape.

`inventory.tpl`

```
1  {% for g in groups ~}
2    [${g.role}]
3    {% for name, ip in g.hosts ~}
4      ${name} ansible_host=${ip}
5    {% endfor ~}
6
7  {% endfor ~}
8  [all:vars]
9  ansible_user=${ci_user}
10 ansible_ssh_private_key_file=${private_key}
11 ansible_python_interpreter=/usr/bin/python3
12 {% if jump_host != null ~}
13 ansible_ssh_common_args='-o ProxyJump=${jump_host}'
14 {% endif ~}
```

5.1.3. Ajouts dans cloud-init et Ansible

Le snippet cloud-init est désormais le même pour toutes les VM : il crée l'utilisateur qu'Ansible utilisera pour se connecter (sudo sans mot de passe, clé SSH) et installe Python.

user-data.yml.tpl

```
1  users:
2    - default
3    - name: ${ci_user}
4      groups: [sudo]
5    [...]
6    sudo: ["ALL=(ALL) NOPASSWD:ALL"]
7    ssh_authorized_keys:
8      - ${ssh_public_key}
9  [...]
10 packages:
11   - vim
12   - curl
13   - python3
14   - python3-apt
```

Le dossier `ansible/` contient `ansible.cfg` (inventaire par défaut `inventory.ini`, donc pas d'option `-i` ; clés d'hôte non vérifiées, les VM étant recrées pendant le TP), `requirements.yml` (collections nécessaires), le playbook `site.yml` et les rôles. Le premier play de `site.yml` s'applique à toutes les VM : il attend la connexion SSH puis la fin de cloud-init, pour qu'Ansible et cloud-init ne se disputent pas `apt`. Les plays suivants ciblent un groupe de l'inventaire et lui appliquent le rôle du même nom, avec les droits root (`become`). Tant que les groupes `database` et `haproxy` n'ont pas de VM, Ansible ignore leurs plays (`skipping: no hosts matched`).

ansible/site.yml

```
1  - name: Attendre que les VM soient prêtes
2    hosts: all
3    gather_facts: false
4    tasks:
5      - name: Attendre la connexion SSH
6        ansible.builtin.wait_for_connection:
7          timeout: 300
8      [...]
9      - name: Attendre la fin de cloud-init
10       ansible.builtin.command: cloud-init status --wait
11      [...]
12  [...]
13  - name: Configurer les serveurs WordPress
14    hosts: wordpress
15    become: true
16    roles: [wordpress]
```

Un rôle regroupe, dans des dossiers aux noms imposés par Ansible, tout ce qui configure un type de serveur :

Dossier du rôle	Contenu
tasks/main.yml	les tâches, dans l'ordre d'exécution
handlers/main.yml	actions déclenchées seulement si une tâche a modifié quelque chose (recharger nginx)
templates/	fichiers de configuration Jinja2, remplis avec les variables
defaults/, vars/	valeurs des variables du rôle

Dans tasks/main.yml du rôle wordpress, chaque tâche décrit un état voulu (paquet présent, fichier de tel contenu et de tels droits) et n'agit que s'il n'est pas déjà atteint. Le module apt installe nginx, php-fpm et les extensions PHP de WordPress. Le service php-fpm porte le numéro de la version de PHP installée (php8.4-fpm, etc.) : le rôle le déduit du binaire présent dans /usr/sbin plutôt que de le fixer en dur.

WordPress n'est téléchargé que s'il n'est pas déjà installé : l'archive est récupérée avec l'empreinte SHA1 publiée par wordpress.org (le téléchargement échoue si elle diffère), puis extraite dans le dossier du site. Un second passage ne touche donc pas à wp-content, qui contient les fichiers téléversés.

ansible/roles/wordpress/tasks/main.yml

```
1 - name: Télécharger et extraire WordPress
2   when: not wordpress_installed.stat.exists
3   block:
4     [...]
5   - name: Télécharger l'archive (empreinte SHA1 publiée par wordpress.org)
6     ansible.builtin.get_url:
7       url: "{{ wordpress_archive_url }}"
8     [...]
9     checksum: "sha1:{{ wordpress_archive_url }}.sha1"
10    [...]
11  - name: Extraire WordPress dans le dossier du site
12    ansible.builtin.unarchive:
13      [...]
14    extra_opts: [--strip-components=1]
```

wp-config.php est généré depuis un modèle Jinja2. Le fichier appartient à root et n'est lisible que par le groupe www-data (il contient le mot de passe) ; php -l le contrôle avant son installation, et diff: false empêche --diff d'afficher le mot de passe.

ansible/roles/wordpress/tasks/main.yml

```
1 - name: Installer wp-config.php
2   ansible.builtin.template:
3     src: wp-config.php.j2
4     dest: "{{ wordpress_webroot }}/wp-config.php"
5     owner: root
6     group: www-data
7     mode: "0640"
8     validate: /usr/bin/php -l %s
9     diff: false
```

Dans le modèle, les identifiants viennent des variables d'Ansible. Chacune des huit clés et sels est le sha512 du mot de passe suivi de son nom : ils sont identiques sur tous les nœuds, sans être stockés nulle part. WP_HOME et WP_SITEURL sont déduits du protocole et de l'en-tête Host de la requête : le site fonctionne quelle que soit l'adresse utilisée pour l'atteindre (nœud ou, plus tard, HAProxy).

ansible/roles/wordpress/templates/wp-config.php.j2

```
1  define( 'DB_NAME',      '{{ wp_db_name }}' );
2  define( 'DB_USER',      '{{ wp_db_user }}' );
3  define( 'DB_PASSWORD',  '{{ wp_db_password }}' );
4  define( 'DB_HOST',      '{{ wordpress_db_host }}' );
5  [...]
6  {% for key in wordpress_salt_names %}
7  define( '{{ key }}', '{{ (wp_db_password ~ ':' ~ key) | hash('sha512') }}' );
8  {% endfor %}
9  [...]
10 if ( isset( $_SERVER['HTTP_HOST'] ) ) {
11     [...]
12     define( 'WP_HOME',    $wp_scheme . '://' . $_SERVER['HTTP_HOST'] );
13     define( 'WP_SITEURL', $wp_scheme . '://' . $_SERVER['HTTP_HOST'] );
14 }
```

Le site nginx est un modèle lui aussi. Quand il change, sa tâche « notifie » le handler Recharger nginx (module service, state: reloaded) : nginx est rechargé une seule fois, au moment du flush_handlers, et seulement si quelque chose a changé. nginx -t contrôle la configuration avant ce rechargement.

ansible/roles/wordpress/tasks/main.yml

```
1  - name: Installer le site nginx
2    ansible.builtin.template:
3      src: nginx-wordpress.conf.j2
4      dest: /etc/nginx/sites-available/wordpress
5      [...]
6    notify: Recharger nginx
7  [...]
8  - name: Contrôler la configuration de nginx
9    ansible.builtin.command: nginx -t
10   changed_when: false
11  [...]
12  - name: Appliquer la configuration
13    ansible.builtin.meta: flush_handlers
```

Dans le modèle du site, `try_files` renvoie les URL propres vers `index.php`, les fichiers cachés et tout PHP dans `wp-content/uploads` sont interdits, `/healthz` répond sans passer par PHP, `/readyz` exécute la sonde décrite plus bas, et les scripts PHP sont confiés à `php-fpm` par son socket, celui de la version installée.

ansible/roles/wordpress/templates/nginx-wordpress.conf.j2

```
1  server {
2      [...]
3      location = /healthz {
4          access_log off;
5          default_type text/plain;
6          return 200 "ok\n";
7      }
8
9      location = /readyz {
10         access_log off;
11         include fastcgi_params;
12         fastcgi_param SCRIPT_FILENAME /var/www/healthz/readyz.php;
13         fastcgi_pass unix:/run/php/php{{ wordpress_php_version }}-fpm.sock;
14     }
15     [...]
16     location / {
17         try_files $uri $uri/ /index.php?$args;
18     }
19     [...]
20 }
```

Deux contrôles vérifient ensuite le nœud : `/healthz` (nginx répond) et `/readyz`, une sonde PHP qui répond 200 si la base accepte le compte WordPress, 503 sinon. Avant le déploiement de la base, 503 est admis : la valeur par défaut `wordpress_readyz_status` vaut `[200, 503]` tant que le groupe `database` est vide, puis `[200]`.

ansible/roles/wordpress/tasks/main.yml

```
1 - name: Vérifier que nginx répond (/healthz)
2   ansible.builtin.uri:
3     url: http://127.0.0.1/healthz
4     status_code: 200
5   [...]
6 - name: Vérifier que la base partagée est joignable (/readyz)
7   ansible.builtin.uri:
8     url: http://127.0.0.1/readyz
9     status_code: "{{ wordpress_readyz_status }}"
```

ansible/roles/wordpress/defaults/main.yml

```
1 wordpress_readyz_status: >-
2   {{ [200] if groups['database'] | default([]) | length > 0 else [200, 503] }}
```

La sonde `readyz.php` est volontairement indépendante de WordPress : elle lit les identifiants dans `wp-config.php` sans charger WordPress, donc répond vite et ne dépend ni des plugins ni des mises à jour. Elle est installée hors du dossier de WordPress (`/var/www/healthz`) : une mise à jour de WordPress ne l'écrase pas.

ansible/roles/wordpress/templates/readyz.php.j2

```
1  mysqli_report( MYSQLI_REPORT_OFF );
2  $link = mysqli_init();
3  mysqli_options( $link, MYSQLI_OPT_CONNECT_TIMEOUT, 2 );
4  [...]
5  if ( ! mysqli_query( $link, 'SELECT 1' ) ) {
6      not_ready( 'requête SQL refusée' );
7  }
8  mysqli_close( $link );
9
10 echo "ok\n";
```

5.1.4. Vérification

À ce stade, la base n'existe pas encore : nginx et php-fpm fonctionnent, mais la page d'accueil de WordPress renvoie une erreur 500 (« Error establishing a database connection ») et `/readyz` répond 503. Ce résultat est attendu.

```
"sti-wp01" = {
  "ip" = "10.0.0.140"
  "mac" = "FA:52:F9:C2:3E:BC"
  "role" = "wordpress"
  "vmid" = 120
```

Figure 1 : Sortie vms de terraform apply : sti-wp01 (IP 10.0.0.140, MAC, vmid 120)

sti-wp01 (Uptime: 00:19:52)

 Status	running
 HA State	none
 Node	pve
 CPU usage	0.78% of 2 CPUs
 Memory usage	38.74% (793.34 MiB of 2.00 GiB)
 Host memory usage	793.34 MiB
 Bootdisk size	40.00 GiB
 IPs	10.0.0.140 fe80::f852:f9ff:fec2:3ebc
More	

Figure 2 : Résumé de sti-wp01 dans Proxmox : VM démarrée, IP fixe 10.0.0.140 inchangée

```
Last login: Tue Sep 29 12:43:05 2026 from 10.0.0.0
root@sti-wp01:~# systemctl status nginx 'php*-fpm' --no-pager
Unit nginx.service could not be found.
root@sti-wp01:~# sudo systemctl status nginx 'php*-fpm' --no-pager
Unit nginx.service could not be found.
root@sti-wp01:~# nginx
-bash: nginx: command not found
root@sti-wp01:~#
```

Figure 3 : sti-wp01 avant ansible-playbook : nginx n'est pas encore installé

```
root@sti-wp01:~# sudo systemctl status nginx 'php*-fpm' --no-pager
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-10-01 16:25:44 UTC; 1min 11s ago
     Docs: man:nginx(8)
  Process: 12130 ExecReload=/usr/sbin/nginx -g daemon on; master_process on; -s
 reload (code=exited, status=0/SUCCESS)
    Main PID: 8724 (nginx)
      Tasks: 3 (limit: 2315)
     Memory: 2.5M
        CPU: 60ms
    CGroup: /system.slice/nginx.service
            └─ 8724 "nginx: master process /usr/sbin/nginx -g daemon on; master_...
               └─ 12131 "nginx: worker process"
                  └─ 12132 "nginx: worker process"

Oct 01 16:25:44 sti-wp01 systemd[1]: Starting nginx.service - A high performa...r...
Oct 01 16:25:44 sti-wp01 systemd[1]: Started nginx.service - A high performan...ver.
Oct 01 16:26:28 sti-wp01 systemd[1]: Reloading nginx.service - A high perform...r...
Oct 01 16:26:28 sti-wp01 nginx[12130]: 2026/10/01 16:26:28 [notice] 12130#121...rted
Oct 01 16:26:28 sti-wp01 systemd[1]: Reloaded nginx.service - A high performa...ver.
Hint: Some lines were ellipsized, use -l to show in full.
root@sti-wp01:~# nginx
.ansible/      .bashrc      .local/      .ssh/
.bash_history  .lessshst   .profile     keyboard.sh
root@sti-wp01:~# nginx
.ansible/      .bashrc      .local/      .ssh/
.bash_history  .lessshst   .profile     keyboard.sh
root@sti-wp01:~# nginx
.ansible/      .bashrc      .local/      .ssh/
.bash_history  .lessshst   .profile     keyboard.sh
root@sti-wp01:~# nginx
nginx: [emerg] bind() to 0.0.0.0:80 failed (98: Address already in use)
nginx: [emerg] bind() to 0.0.0.0:80 failed (98: Address already in use)
nginx: [emerg] bind() to 0.0.0.0:80 failed (98: Address already in use)
nginx: [emerg] bind() to 0.0.0.0:80 failed (98: Address already in use)
^C
root@sti-wp01:~#
```

Figure 4 : sti-wp01 après ansible-playbook : service nginx actif

```
root@sti-wp01:~# curl -s http://127.0.0.1/healthz
ok
root@sti-wp01:~# sudo nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
root@sti-wp01:~#
```

Figure 5 : sti-wp01 : sonde /healthz (ok) et configuration nginx valide (nginx -t)

```
root@sti-wp01:~# ls /var/www/wordpress
index.php      wp-blog-header.php  wp-cron.php      wp-mail.php
license.txt    wp-comments-post.php wp-includes       wp-settings.php
readme.html    wp-config-sample.php wp-links-opml.php wp-signup.php
wp-activate.php wp-config.php        wp-load.php       wp-trackback.php
wp-admin        wp-content           wp-login.php      xmlrpc.php
root@sti-wp01:~# ls -l /var/www/wordpress/wp-config.php
-rw-r----- 1 root www-data 3240 Oct 1 16:26 /var/www/wordpress/wp-config.php
root@sti-wp01:~# sudo grep -E "DB_(NAME|USER|HOST)" /var/www/wordpress/wp-config.p
hp
define( 'DB_NAME',      'wordpress' );
define( 'DB_USER',      'wordpress' );
define( 'DB_HOST',      '10.0.0.170' );
root@sti-wp01:~#
```

Figure 6 : sti-wp01 : fichiers de WordPress, droits de wp-config.php et paramètres de base de données

5.2. Duplication de la VM

sti-wp02 (vmid 121, 10.0.0.160) est une copie de sti-wp01 : même modèle, même cloud-init, même rôle Ansible. Seuls changent son nom, son adresse MAC et son IP fixe.

5.2.1. Déroulement

1. Le `main.tf` de la première partie décrit une seule VM, dont les ressources portent des noms fixes (`debian_clone`, `vmmac`). Copier le fichier donnerait deux versions du même code à maintenir.
2. Les VM sont donc décrites par une table `vms` (nom, rôle, vmid, IP), et chaque ressource est créée avec `for_each = var.vms` : ajouter une VM revient à ajouter une ligne.
3. Les ressources existantes changent d'adresse dans l'état Terraform (`debian_clone` devient `vm["sti-wp01"]`). Des blocs `moved` l'indiquent : Terraform renomme les entrées sans rien détruire, et `sti-wp01` garde sa VM, sa MAC et son IP.
4. Pour chaque VM, Terraform enchaîne : adresse MAC, réservation Kea (IP fixe), envoi du snippet sur Proxmox, puis clonage. Réservation et snippet existent avant le premier démarrage : la VM obtient son IP fixe dès son premier boot.
5. Côté Ansible, rien à écrire : la table `vms` alimente aussi l'inventaire, donc `sti-wp02` rejoint le groupe `wordpress` et le play existant s'applique aux deux nœuds. Ils sont identiques par construction : mêmes tâches, mêmes variables.
6. Les deux nœuds reçoivent les mêmes clés et sels WordPress, calculés à partir du mot de passe : une session ouverte sur un nœud reste valide sur l'autre.
7. `ansible-playbook site.yml --limit sti-wp02` ne configure que la nouvelle VM ; sans `--limit`, `sti-wp01` est parcourue aussi et rien n'y change.

5.2.2. Ajouts dans les fichiers Terraform

La table `vms` de `var.tf` associe à chaque nom de VM son rôle, son vmid et son IP. `dhcp_label` (facultatif) conserve le libellé de la réservation d'origine de `sti-wp01`, pour ne rien modifier côté OPNsense. Des validations refusent les noms invalides, les rôles inconnus et les IP ou vmid en double.

```
var.tf
1  variable "vms" {
2    [...]
3    default = {
4      "sti-wp01" = {
5        role      = "wordpress"
6        vmid      = 120
7        ip        = "10.0.0.140"
8      }
9    }
10   "sti-wp02" = {
11     role = "wordpress"
12     vmid = 121
13     ip   = "10.0.0.160"
14   }
15   [...]
16 }
17 [...]
18 }
```

Dans `main.tf`, les ressources de la première partie reçoivent `for_each = var.vms` ; `each.key` est le nom de la VM et `each.value` sa ligne de la table. La MAC reste propre à chaque VM et l'IP de la réservation Kea vient de la table.

```
main.tf
1  resource "macaddress" "vm" {
2    for_each = var.vms
3  }
4
5  resource "opnsense_kea_dhcpv4_reservation" "vm" {
6    for_each = var.vms
7
8    subnet_id   = "4f275fc3-7f2d-4bdb-b8a7-eb794075aa7f"
9    ip_address  = each.value.ip
10   mac_address = lower(macaddress.vm[each.key].address)
11   hostname    = coalesce(each.value.dhcp_label, each.key)
12   description = "Static IP for ${coalesce(each.value.dhcp_label, each.key)}"
13 }
```

Le clone prend son vmid et son tag dans la table, son snippet et sa MAC d'après le nom de la VM, et ne démarre qu'une fois le snippet envoyé et la réservation créée.

```
main.tf
1  resource "proxmox_vm_qemu" "vm" {
2    for_each = var.vms
3
4    name      = each.key
5    [...]
6    depends_on = [terraform_data.upload_snippet, opnsense_kea_dhcpv4_reservation.vm]
7    [...]
8    cicustom = "user=local:snippets/${local.snippet_file[each.key]}"
9    tags      = each.value.role
10
11   vmid      = each.value.vmid
12   [...]
13   network {
14     [...]
15     macaddr = upper(macaddress.vm[each.key].address)
16   }
17 }
```

La migration sans destruction repose sur les blocs `moved`. Ils sont chaînés pour couvrir l'état de départ (ressource unique) et l'état intermédiaire ; des blocs équivalents existent pour la MAC, la réservation Kea et le snippet.

```
main.tf
1 moved {
2   from = proxmox_vm_qemu.debian_clone
3   to   = proxmox_vm_qemu.wp["sti-wp01"]
4 }
5
6 moved {
7   from = proxmox_vm_qemu.wp["sti-wp01"]
8   to   = proxmox_vm_qemu.vm["sti-wp01"]
9 }
```

Enfin, le snippet de chaque VM est calculé une seule fois à partir du même modèle, avec le nom de la VM comme nom d'hôte ; une sortie `vms` résume le rôle, l'IP, la MAC et le `vmid` de chaque VM.

```
main.tf
1   user_data = {
2     for name, vm in var.vms : name => templatefile("${path.module}/user-data.yml.tpl", {
3       hostname      = name
4       [...]
5     })
6   }
7
8   snippet_file = { for name in keys(var.vms) : name => "${name}-user-data.yml" }
9   [...]
10  output "vms" {
11    [...]
12    value = {
13      for name, vm in var.vms : name => {
14        role = vm.role
15        ip   = vm.ip
16        mac  = upper(macaddress.vm[name].address)
17        vmid = vm.vmid
18      }
19    }
20  }
```

5.2.3. Ajouts dans cloud-init et Ansible

Aucun nouveau rôle ni nouvelle tâche. Côté cloud-init, le nom d'hôte n'est plus écrit en dur (il valait `sti-web01` en première partie) mais reçu de Terraform, et `manage_etc_hosts` garde `/etc/hosts` cohérent avec lui.

```
user-data.yml.tpl
1 #cloud-config
2 hostname: ${hostname}
3 manage_etc_hosts: true
```

Côté Ansible, seul l'inventaire généré change : sti-wp02 apparaît dans le groupe wordpress, et le play existant l'atteint sans modification.

ansible/inventory.ini (généré)

```
1 [wordpress]
2 sti-wp01 ansible_host=10.0.0.140
3 sti-wp02 ansible_host=10.0.0.160
```

5.2.4. Vérification

```
"sti-wp02" = {
  "ip" = "10.0.0.160"
  "mac" = "86:F6:46:3A:FF:CE"
  "role" = "wordpress"
  "vmid" = 121
}
```

Figure 7 : Sortie vms de terraform apply : sti-wp02 (IP 10.0.0.160, MAC, vmid 121)

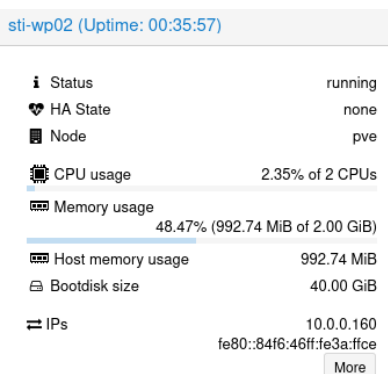


Figure 8 : Résumé de sti-wp02 dans Proxmox : VM démarrée, IP fixe 10.0.0.160

```
root@pve:~# ssh root@10.0.0.160
The authenticity of host '10.0.0.160 (10.0.0.160)' can't be established.
ED25519 key fingerprint is SHA256:MYuk2GT6gwMrx+Yu04MEglXXbuW0UI/LjR7cf5drpEE.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '10.0.0.160' (ED25519) to the list of known hosts.
root@10.0.0.160's password:
Linux sti-wp02 6.1.0-53-amd64 #1 SMP PREEMPT_DYNAMIC Debian 6.1.187-1 (2026-09-07)
x86_64

The programs included with the Debian GNU/Linux system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Debian GNU/Linux comes with ABSOLUTELY NO WARRANTY, to the extent
permitted by applicable law.
Last login: Tue Sep 29 12:43:05 2026 from 10.0.0.0
root@sti-wp02:~# nginx -t
nginx: the configuration file /etc/nginx/nginx.conf syntax is ok
nginx: configuration file /etc/nginx/nginx.conf test is successful
root@sti-wp02:~# systemctl status nginx.service
● nginx.service - A high performance web server and a reverse proxy server
   Loaded: loaded (/lib/systemd/system/nginx.service; enabled; preset: enabled)
   Active: active (running) since Thu 2026-10-01 16:25:46 UTC; 11min ago
     Docs: man:nginx(8)
  Process: 11984 ExecReload=/usr/sbin/nginx -g daemon on; master_process on; -s
 Main PID: 8580 (nginx)
    Tasks: 3 (limit: 2315)
   Memory: 2.5M
      CPU: 239ms
   CGroup: /system.slice/nginx.service
           └─ 8580 "nginx: master process /usr/sbin/nginx -g daemon on; master_
              11985 "nginx: worker process"
              11986 "nginx: worker process"

Oct 01 16:25:46 sti-wp02 systemd[1]: Starting nginx.service - A high performance
Oct 01 16:25:46 sti-wp02 systemd[1]: Started nginx.service - A high performance
Oct 01 16:26:27 sti-wp02 systemd[1]: Reloading nginx.service - A high performance
Oct 01 16:26:27 sti-wp02 nginx[11984]: 2026/10/01 16:26:27 [notice] 11984#11984:
Oct 01 16:26:27 sti-wp02 systemd[1]: Reloaded nginx.service - A high performance
root@sti-wp02:~# curl -s http://127.0.0.1/healthz
ok
root@sti-wp02:~# sudo grep -E "DB_(NAME|USER|HOST)" /var/www/wordpress/wp-config.p
hp
define( 'DB_NAME',          'wordpress' );
define( 'DB_USER',          'wordpress' );
define( 'DB_HOST',          '10.0.0.170' );
root@sti-wp02:~#
```

Figure 9 : sti-wp02 : mêmes vérifications que pour sti-wp01 (nginx, /healthz, paramètres de base de données)

5.3. Nouvelle VM MariaDB

sti-db01 (vmid 122, 10.0.0.170) héberge la base partagée par les deux nœuds WordPress. Le cours demande une seule base, un compte dédié à WordPress et un accès limité au réseau web.

5.3.1. Déroulement

1. Une ligne de rôle database est ajoutée à la table vms : MAC, réservation Kea, snippet et VM sont créés par les for_each existants, sans nouvelle ressource. Le cloud-init est celui des autres VM ; l'inventaire généré reçoit un groupe database non vide.
2. Un nouveau rôle, database, installe MariaDB. Dans site.yml, son play passe avant celui de WordPress : quand les nœuds sont configurés, la base est prête. Dès que le groupe database n'est plus vide, le rôle wordpress exige que /readyz réponde 200.
3. Sous Debian, MariaDB n'écoute que sur 127.0.0.1 : le rôle dépose un fragment de configuration qui l'ouvre au réseau, désactive la résolution DNS inverse (les comptes sont définis par adresse IP) et fixe l'encodage utf8mb4. MariaDB n'est redémarrée que si ce fichier a changé.
4. Le compte WordPress n'existe que pour le réseau autorisé (db_allowed_cidrs, par défaut 10.0.0.0/24, le « VLAN web » du cours). Les comptes anonymes et la base test sont supprimés.
5. La base et le compte sont décrits avec les modules mysql_db et mysql_user plutôt qu'avec des commandes SQL : le résultat est le même à chaque passage, le mot de passe ne figure dans aucune ligne de commande et reste absent des journaux d'Ansible. Un compte créé pour une source qui n'est plus autorisée est supprimé.
6. DB_HOST n'est pas saisi à la main : Terraform le calcule à partir de la table vms et l'écrit dans les variables d'Ansible, ce qui relie les deux nœuds à 10.0.0.170. Le mot de passe est défini une seule fois (wp_db_password) pour le compte MariaDB et pour wp-config.php.
7. Les nœuds WordPress sont reconfigurés par un simple nouveau passage d'ansible-playbook : leur wp-config.php reçoit la nouvelle adresse de la base, rien d'autre ne change. Aucune VM n'est arrêtée ni recrée (cloud-init, lui, ne rejoue pas sa configuration après le premier démarrage).
8. La règle OPNsense du cours (MariaDB, port 3306, depuis le réseau web uniquement) est prévue mais facultative : les VM partagent ici le même sous-réseau, ce trafic ne traverse pas OPNsense et l'isolation repose sur les comptes MariaDB.

5.3.2. Ajouts dans les fichiers Terraform

Dans var.tf, la nouvelle VM et deux variables : les réseaux autorisés à se connecter à la base, et l'interface OPNsense de la règle facultative (null : aucune règle).

```
var.tf
1  "sti-db01" = {
2      role = "database"
3      vmid = 122
4      ip   = "10.0.0.170"
5  }
6  [...]
7  variable "db_allowed_cidrs" {
8      [...]
9      default    = ["10.0.0.0/24"]
10     [...]
11 }
12
13 variable "db_firewall_interface" {
14     [...]
15     default    = null
16 }
```

Dans `main.tf`, l'hôte de la base est déduit de la VM de rôle `database` (ou du nom prévu `sti-db01` tant qu'elle n'existe pas, ce qui permettait à l'étape précédente d'écrire déjà un `wp-config.php` complet). Les réseaux autorisés sont convertis au format adresse/masque de MariaDB. Les deux valeurs rejoignent les variables écrites pour Ansible.

```
main.tf
1  locals {
2    database_ips = [for vm in values(var.vms) : vm.ip if vm.role == "database"]
3    [...]
4    db_host = coalesce(var.wp_db_host, one(local.database_ips), "sti-db01")
5    [...]
6    db_allowed_hosts = [for c in var.db_allowed_cidrs : "${cidrhost(c, 0)}/${cidrnetmask(c)}"]
7    [...]
8    ansible_vars = {
9      [...]
10     wordpress_db_host      = local.db_host
11     [...]
12     database_allowed_hosts = local.db_allowed_hosts
13     [...]
14   }
15 }
```

La règle du cours est une ressource OPNsense, créée seulement si `db_firewall_interface` est renseignée.

```
main.tf
1  resource "opnsense_firewall_filter" "db_from_web_only" {
2    [...]
3    filter = {
4      action      = "pass"
5      direction   = "in"
6      ip_protocol = "inet"
7      protocol    = "TCP"
8      source      = { net = each.value }
9      destination = { net = local.database_ips[0], port = "3306" }
10   }
11 }
```

5.3.3. Ajouts dans Ansible

Dans `site.yml`, le play de la base est placé avant celui de WordPress, ce qui fixe l'ordre de déploiement.

```
ansible/site.yml
1 - name: Configurer la base de données
2   hosts: database
3   become: true
4   roles: [database]
5
6 - name: Configurer les serveurs WordPress
7   hosts: wordpress
8   become: true
9   roles: [wordpress]
```

Le rôle database commence par installer MariaDB, puis dépose le fragment de configuration. Cette tâche « notifie » le handler Redémarrer MariaDB, exécuté seulement si le fichier a changé. python3-pymysql est ce qu'utilisent les modules MySQL d'Ansible, fournis par la collection community.mysql (fichier requirements.yml).

ansible/roles/database/tasks/main.yml

```
1 - name: Installer MariaDB (et PyMySQL, qu'utilisent les modules community.mysql)
2   ansible.builtin.apt:
3     name:
4       - mariadb-server
5       - mariadb-client
6       - python3-pymysql
7     state: present
8     [...]
9 - name: Déposer la configuration de MariaDB
10  ansible.builtin.copy:
11    src: mariadb-sti.cnf
12    dest: /etc/mysql/mariadb.conf.d/99-sti.cnf
13    [...]
14  notify: Redémarrer MariaDB
```

Le fragment ouvre MariaDB au réseau, désactive la résolution DNS inverse et fixe l'encodage.

ansible/roles/database/files/mariadb-sti.cnf

```
1 [mysqld]
2 bind-address = 0.0.0.0
3 skip-name-resolve
4 character-set-server = utf8mb4
5 collation-server      = utf8mb4_unicode_ci
```

Les comptes sont ensuite gérés par le socket Unix de MariaDB, où root s'authentifie sans mot de passe sous Debian. Le rôle supprime d'abord la base test et les comptes anonymes (host_all: true : quel que soit leur hôte), puis crée la base de WordPress et son compte pour chaque réseau autorisé ; update_password: always resynchronise le mot de passe à chaque passage.

ansible/roles/database/tasks/main.yml

```
1 - name: Créer la base de WordPress
2   community.mysql.mysql_db:
3     [...]
4   name: "{{ wp_db_name }}"
5   encoding: utf8mb4
6   [...]
7 - name: Créer le compte de WordPress pour chaque réseau autorisé
8   community.mysql.mysql_user:
9     [...]
10  name: "{{ wp_db_user }}"
11  host: "{{ item }}"
12  password: "{{ wp_db_password }}"
13  update_password: always
14  priv: "{{ wp_db_name }}.*:ALL"
15  state: present
16  loop: "{{ database_allowed_hosts }}"
```

Le rôle retire ensuite les accès qui ne sont plus autorisés : il liste les sources actuelles du compte WordPress et supprime celles qui ne figurent pas dans la liste autorisée.

```
ansible/roles/database/tasks/main.yml
1 - name: Lister les sources actuelles du compte WordPress
2   community.mysql.mysql_query:
3     [...]
4     query: SELECT Host AS host FROM mysql.user WHERE User = %s
5     [...]
6     register: database_sources
7     [...]
8 - name: Supprimer le compte des sources non autorisées
9   community.mysql.mysql_user:
10    [...]
11    host: "{{ item }}"
12    state: absent
13  loop: >-
14    {{ database_sources.query_result[0] | map(attribute='host')
15      | difference(database_allowed_hosts) }}
```

Enfin, le rôle vérifie que MariaDB écoute bien sur l'adresse de la VM : l'écoute limitée à 127.0.0.1 est la première cause de l'erreur « Error establishing a database connection », autant échouer ici qu'à l'ouverture du site.

```
ansible/roles/database/tasks/main.yml
1 - name: Vérifier que MariaDB écoute sur l'adresse de la VM
2   ansible.builtin.wait_for:
3     host: "{{ ansible_host }}"
4     port: 3306
5     timeout: 15
```

5.3.4. Vérification

Accès direct aux nœuds. Les captures qui joignent un nœud directement ne sont possibles que jusqu'à l'étape suivante : une fois HAProxy déployé, les nœuds refusent (403) tout appelant autre que lui.

```
vms = {
  "sti-db01" = {
    "ip" = "10.0.0.170"
    "mac" = "66:56:78:B7:16:13"
    "role" = "database"
    "vmid" = 122
  }
}
```

Figure 10 : Sortie vms de terraform apply : sti-db01 (IP 10.0.0.170, MAC, vmid 122)

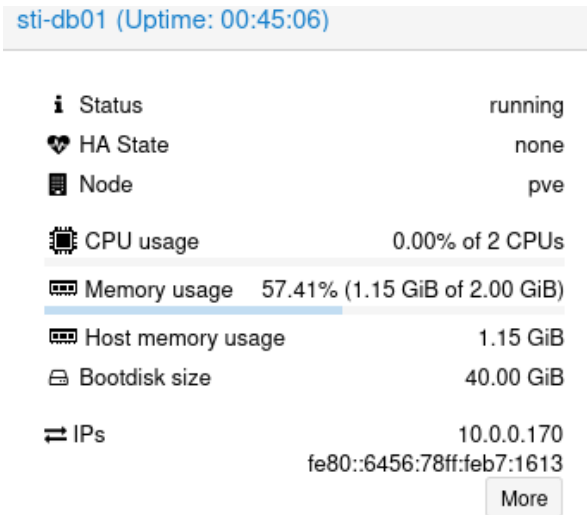


Figure 11 : Résumé de sti-db01 dans Proxmox : VM démarrée, IP fixe 10.0.0.170

```

root@sti-db01:~# sudo ss -ltn 'sport = :3306'
State      Recv-Q      Send-Q      Local Address:Port      Peer Address:Port      Process
LISTEN     0            80          0.0.0.0:3306             0.0.0.0:*
root@sti-db01:~# sudo mariadb -e "SELECT user,host FROM mysql.user"
+-----+-----+
| User           | Host           |
+-----+-----+
| wordpress      | 10.0.0.0/255.255.255.0 |
| mariadb.sys    | localhost      |
| mysql          | localhost      |
| root           | localhost      |
+-----+-----+
root@sti-db01:~# █

```

Figure 12 : sti-db01 : MariaDB à l'écoute sur le port 3306, compte wordpress limité à 10.0.0.0/255.255.255.0, sans compte anonyme

5.4. Nouvelle VM proxy HAProxy

sti-lb01 (vmid 123, 10.0.0.180) est l'unique point d'entrée de la plateforme : il répartit les requêtes HTTP entre les deux nœuds WordPress et écarte automatiquement un nœud en panne.

5.4.1. Déroulement

1. Une ligne de rôle haproxy est ajoutée à la table vms ; l'inventaire généré reçoit un groupe haproxy. Le rôle du même nom installe haproxy et rsyslog (sans lui, les journaux d'HAProxy sont perdus sous Debian).
2. haproxy.cfg est généré par un modèle Jinja2 : les lignes server du backend viennent du groupe wordpress de l'inventaire, aucune IP n'est écrite en dur. Passer de 2 à N nœuds revient à ajouter des lignes à la table vms, puis à relancer terraform apply et ansible-playbook.
3. Le frontend écoute sur le port 80 et répartit en round-robin. La sonde /readyz de chaque nœud vérifie php-fpm et la connexion à MariaDB : un nœud est retiré après 3 échecs (environ 6 s), remis après 2 succès, avec une montée en charge progressive.
4. Une configuration invalide ne doit jamais interrompre le service : le fichier est validé par haproxy -c avant de remplacer l'ancien, puis HAProxy est rechargé à chaud (sans couper les connexions), seulement si le fichier a changé. Si le port 80 n'est pas à l'écoute à la fin, le rôle redémarre HAProxy une fois.
5. Comme le demande le cours, les nœuds WordPress ne répondent plus qu'à HAProxy (403 pour tout autre appelant, sauf eux-mêmes pour leurs sondes locales). Les adresses d'HAProxy viennent de l'inventaire : le rôle wordpress réécrit donc le site nginx des nœuds et le recharge. nginx lit l'IP réelle du client dans X-Forwarded-For, mais seulement si la requête vient d'HAProxy, qui écrase les en-têtes envoyés par le client.
6. Une requête de lecture (GET, HEAD, OPTIONS) est rejouée sur l'autre nœud si le premier coupe la connexion ou répond 502 ou 503. Les autres méthodes (POST...) ne le sont que si la connexion a échoué avant l'envoi de la requête, pour ne jamais exécuter deux fois un formulaire.
7. L'en-tête X-Served-By indique quel nœud a répondu (le round-robin devient visible) et une page d'état en lecture seule est servie sur le port 8404 aux réseaux autorisés.
8. Le rôle se termine par deux contrôles : la plateforme répond à travers HAProxy, puis l'état de chaque nœud, lu sur la page d'état, est affiché dans la sortie d'Ansible.

5.4.2. Ajouts dans les fichiers Terraform

Dans var.tf, la nouvelle VM et deux variables : la restriction des nœuds à HAProxy (active par défaut, désactivable pour dépanner) et les réseaux autorisés à consulter la page d'état (liste vide : page désactivée). Des validations imposent une seule VM haproxy, et au moins un nœud wordpress à répartir.

```
var.tf
1  "sti-lb01" = {
2      role = "haproxy"
3      vmid = 123
4      ip   = "10.0.0.180"
5  }
6  [...]
7  variable "wp_only_from_proxy" {
8      [...]
9      default = true
10 }
11
12 variable "lb_stats_cidrs" {
13     [...]
14     default = ["10.0.0.0/24"]
15     [...]
16 }
```

Dans `main.tf`, l'adresse du répartiteur sert à la sortie `lb_url` ; la restriction et les réseaux de la page d'état rejoignent les variables écrites pour Ansible. Terraform ne construit plus le backend d'HAProxy : il vient de l'inventaire.

```
main.tf
1  locals {
2    [...]
3    haproxy_ips = [for vm in values(var.vms) : vm.ip if vm.role == "haproxy"]
4    lb_ip       = one(local.haproxy_ips)
5    [...]
6    ansible_vars = {
7      [...]
8      wordpress_only_from_proxy = var.wp_only_from_proxy
9      [...]
10     haproxy_stats_cidrs       = var.lb_stats_cidrs
11   }
12 }
13 [...]
14 output "lb_url" {
15   [...]
16   value       = local.lb_ip == null ? null : "http://${local.lb_ip}/"
17 }
```

Au terme des quatre étapes, Terraform produit l'inventaire et les variables suivants ; le mot de passe de la base est écrit à part, dans `secrets.yml`.

```
ansible/inventory.ini (génééré)
1  [database]
2  sti-db01 ansible_host=10.0.0.170
3
4  [wordpress]
5  sti-wp01 ansible_host=10.0.0.140
6  sti-wp02 ansible_host=10.0.0.160
7
8  [haproxy]
9  sti-lb01 ansible_host=10.0.0.180
10
11 [all:vars]
12 ansible_user=admin
13 ansible_ssh_private_key_file=~/.ssh/id_ed25519
14 ansible_python_interpreter=/usr/bin/python3
```

ansible/group_vars/all/terraform.yml (généré)

```
1 "database_allowed_hosts":
2 - "10.0.0.0/255.255.255.0"
3 "haproxy_stats_cidrs":
4 - "10.0.0.0/24"
5 "wordpress_db_host": "10.0.0.170"
6 "wordpress_only_from_proxy": true
7 "wordpress_version": "7.1.2"
8 "wp_db_name": "wordpress"
9 "wp_db_user": "wordpress"
```

5.4.3. Ajouts dans Ansible

Le dernier play de site.yml applique le rôle haproxy au groupe du même nom.

ansible/site.yml

```
1 - name: Configurer le load balancer
2   hosts: haproxy
3   become: true
4   roles: [haproxy]
```

Dans le rôle haproxy, validate fait contrôler le fichier par haproxy -c avant qu'il remplace l'actuel : en cas d'erreur, la tâche échoue et le service continue avec l'ancienne configuration. La tâche notifie le handler Recharger HAProxy (module service, state: reloaded), qui recharge HAProxy à chaud, sans couper les connexions, seulement si le fichier a changé.

ansible/roles/haproxy/tasks/main.yml

```
1 - name: Installer haproxy.cfg (validé par « haproxy -c » avant remplacement)
2   ansible.builtin.template:
3     src: haproxy.cfg.j2
4     dest: /etc/haproxy/haproxy.cfg
5     [...]
6     validate: haproxy -c -f %s
7     notify: Recharger HAProxy
```

Si HAProxy tourne sans écouter sur le port 80 (par exemple avec la configuration d'origine du paquet, qui n'a pas de frontend), un bloc block / rescue le redémarre franchement, une seule fois.

ansible/roles/haproxy/tasks/main.yml

```
1 - name: Vérifier qu'HAProxy écoute sur le port 80
2   block:
3     - name: Attendre l'ouverture du port 80
4       ansible.builtin.wait_for:
5         port: 80
6         timeout: 15
7     rescue:
8       [...]
9     - name: Redémarrer HAProxy
10      ansible.builtin.service:
11        name: haproxy
12        state: restarted
```

Le modèle `haproxy.cfg.j2` contient le frontend, le backend et, en fin de fichier, la page d'état (non reproduite ici). La boucle `{% for %}` écrit une ligne `server` par hôte du groupe `wordpress`, et `hostvars[host].ansible_host` donne son IP fixe.

ansible/roles/haproxy/templates/haproxy.cfg.j2

```
1  frontend http_front
2      bind *:80
3      http-request del-header X-Forwarded-For
4      http-request set-header X-Forwarded-Proto http
5      default_backend wordpress_back
6
7  backend wordpress_back
8      balance roundrobin
9      [...]
10     option httpchk
11     http-check send meth GET uri /readyz ver HTTP/1.1 hdr Host haproxy-healthcheck
12     http-check expect status 200
13     [...]
14     default-server check inter 2s fall 3 rise 2 slowstart 10s
15     [...]
16     retry-on conn-failure empty-response 502 503
17     http-request disable-l7-retry unless METH_GET || METH_OPTIONS
18     [...]
19     http-response set-header X-Served-By %s
20     [...]
21 {% for host in groups['wordpress'] %}
22     server {{ host }} {{ hostvars[host].ansible_host }}:80
23 {% endfor %}
```

Les deux derniers contrôles du rôle vérifient la plateforme de bout en bout : une requête passe par HAProxy jusqu'à un nœud (200, ou 302 vers l'assistant d'installation tant que WordPress n'est pas installé), puis l'état de chaque nœud est lu sur la page d'état locale et affiché dans la sortie d'Ansible (`sti-wp01 : UP`).

ansible/roles/haproxy/tasks/main.yml

```
1  - name: Vérifier qu'au moins un nœud WordPress répond derrière HAProxy
2      ansible.builtin.uri:
3          url: http://127.0.0.1/
4          follow_redirects: none
5          status_code: [200, 301, 302]
6      [...]
7  - name: Lire l'état des nœuds sur la page d'état locale
8      ansible.builtin.uri:
9          url: "http://127.0.0.1:8404/stats;csv"
10         return_content: true
```

Côté nœuds, le rôle wordpress calcule les adresses d'HAProxy depuis l'inventaire (wordpress_proxy_ips, vide tant qu'aucune VM haproxy n'existe). set_real_ip_from ne fait confiance à X-Forwarded-For que pour ces adresses, et toute autre adresse reçoit un 403.

ansible/roles/wordpress/defaults/main.yml

```
1 wordpress_proxy_ips: >-
2   {{ groups['haproxy'] | default([]) | map('extract', hostvars, 'ansible_host')
3     | list }}
4 wordpress_only_from_proxy: true
```

ansible/roles/wordpress/templates/nginx-wordpress.conf.j2

```
1 {% if wordpress_proxy_ips | length > 0 %}
2   [...]
3 {% for ip in wordpress_proxy_ips %}
4   set_real_ip_from {{ ip }};
5 {% endfor %}
6   real_ip_header    X-Forwarded-For;
7 {% endif %}
8 {% if wordpress_only_from_proxy and wordpress_proxy_ips | length > 0 %}
9 {% set proxy_regex = wordpress_proxy_ips | map('regex_escape') | join('|') %}
10  [...]
11  if ($realip_remote_addr !~ ^(127\.0\.0\.1|{{ proxy_regex }})$) {
12    return 403;
13  }
14 {% endif %}
```

Enfin, wp-config.php tient compte de l'en-tête X-Forwarded-Proto envoyé par HAProxy, ce qui prépare un futur HTTPS terminé sur le répartiteur.

5.4.4. Vérification

Outputs:

```
lb_stats_url = "http://10.0.0.180:8404/stats"
lb_url = "http://10.0.0.180/"
vms = {
  "sti-db01" = {
    "ip" = "10.0.0.170"
    "mac" = "66:56:78:B7:16:13"
    "role" = "database"
    "vmid" = 122
  }
  "sti-lb01" = {
    "ip" = "10.0.0.180"
    "mac" = "6E:6E:80:AC:CC:E1"
    "role" = "haproxy"
    "vmid" = 123
  }
  "sti-wp01" = {
    "ip" = "10.0.0.140"
    "mac" = "FA:52:F9:C2:3E:BC"
    "role" = "wordpress"
    "vmid" = 120
  }
  "sti-wp02" = {
    "ip" = "10.0.0.160"
    "mac" = "86:F6:46:3A:FF:CE"
    "role" = "wordpress"
    "vmid" = 121
  }
}
wp_db_host = "10.0.0.170"
```

Figure 13 : Sorties de terraform apply : adresses de la plateforme (lb_url, lb_stats_url, wp_db_host) et table vms des quatre VM

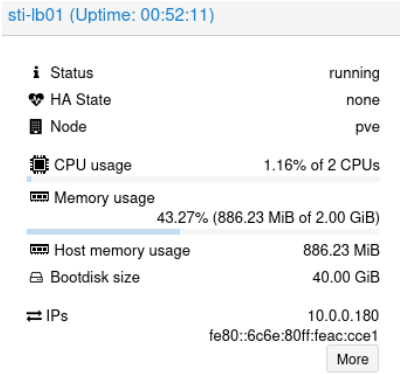


Figure 14 : Résumé de sti-lb01 dans Proxmox : VM démarrée, IP fixe 10.0.0.180

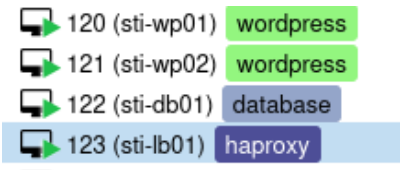


Figure 15 : Les quatre VM dans Proxmox : vmid 120 à 123, démarrées, avec leur étiquette de rôle

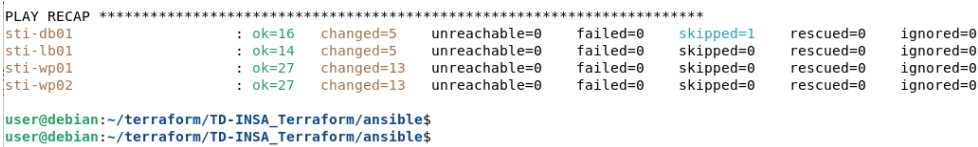


Figure 16 : Bilan PLAY RECAP de ansible-playbook site.yml : quatre VM, aucune injoignable ni en échec

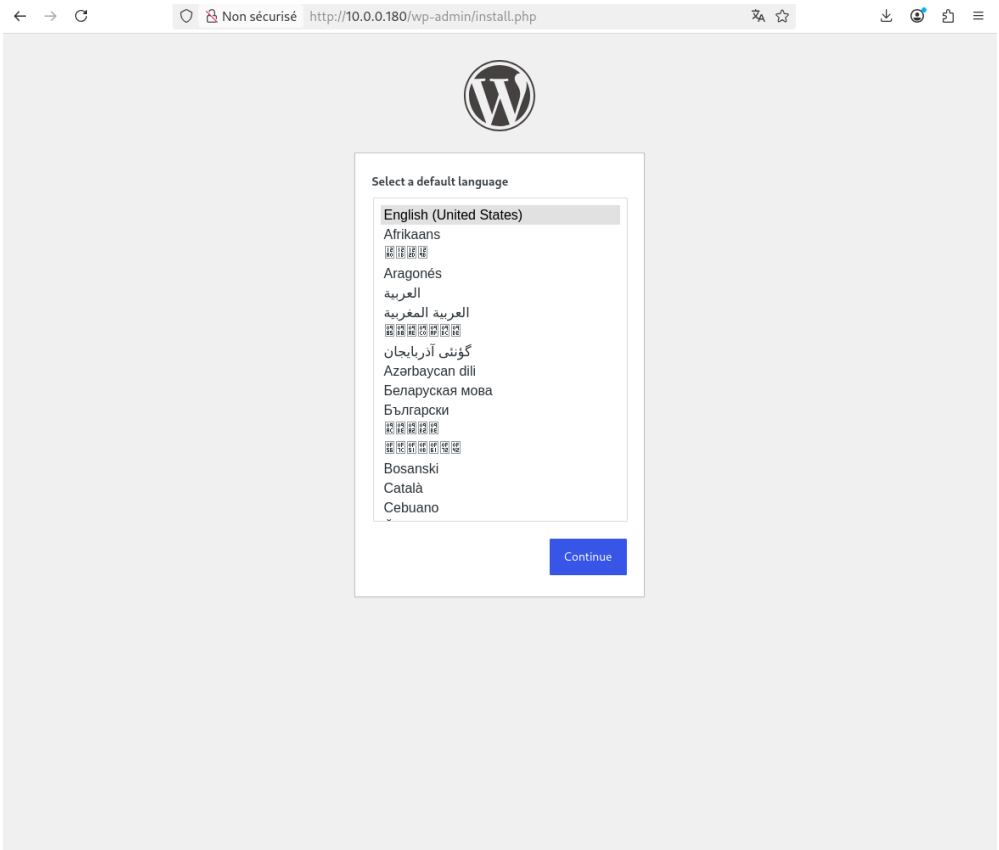


Figure 17 : Assistant d’installation de WordPress, atteint à travers HAProxy (http://10.0.0.180/)

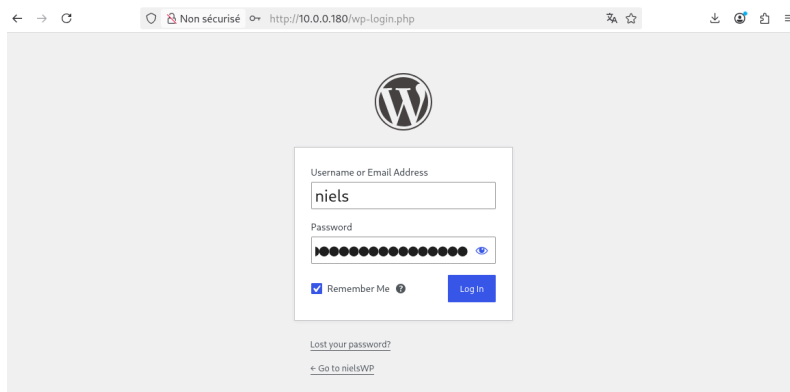


Figure 18 : Page de connexion de WordPress (/wp-login.php) : compte créé à l'installation

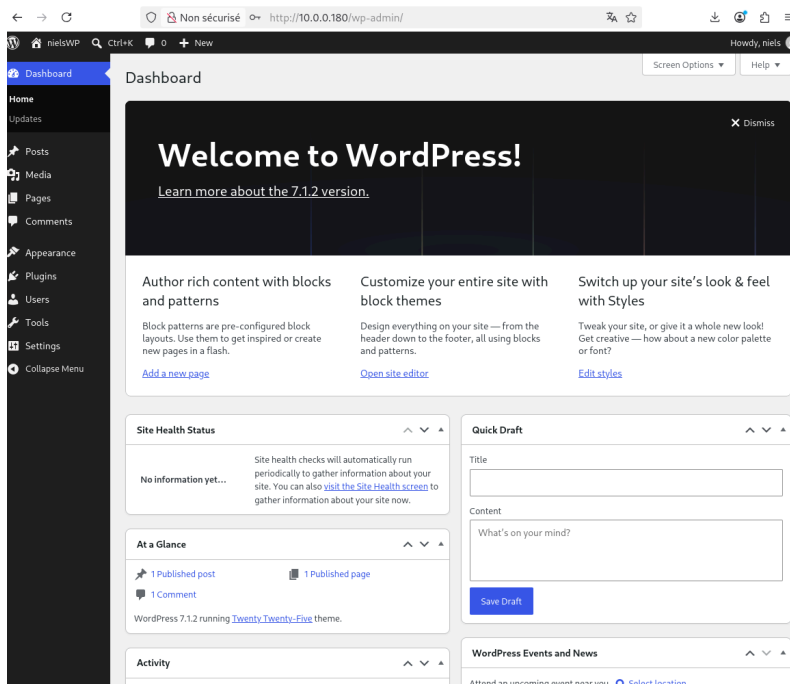


Figure 19 : Tableau de bord de WordPress 7.1.2 (/wp-admin/) : connexion réussie à travers HAProxy

```
root@sti-db01:~# sudo mariadb wordpress -e 'SHOW TABLES'
+-----+
| Tables_in_wordpress |
+-----+
| wp_commentmeta      |
| wp_comments         |
| wp_links            |
| wp_options          |
| wp_postmeta         |
| wp_posts            |
| wp_term_relationships |
| wp_term_taxonomy    |
| wp_termmeta         |
| wp_terms            |
| wp_usermeta         |
| wp_users            |
+-----+
root@sti-db01:~#
```

Figure 20 : Tables WordPress créées dans la base partagée de sti-db01

```
user@debian:~/terraform/TD-INSA_Terraform/ansible$ for i in 1 2 3 4 5 6; do curl
-sI http://10.0.0.180/ | grep -i '^x-served-by'; done
x-served-by: sti-wp01
x-served-by: sti-wp02
x-served-by: sti-wp01
x-served-by: sti-wp02
x-served-by: sti-wp01
x-served-by: sti-wp02
user@debian:~/terraform/TD-INSA_Terraform/ansible$
```

Figure 21 : Répartition de charge : l'en-tête X-Served-By alterne entre sti-wp01 et sti-wp02

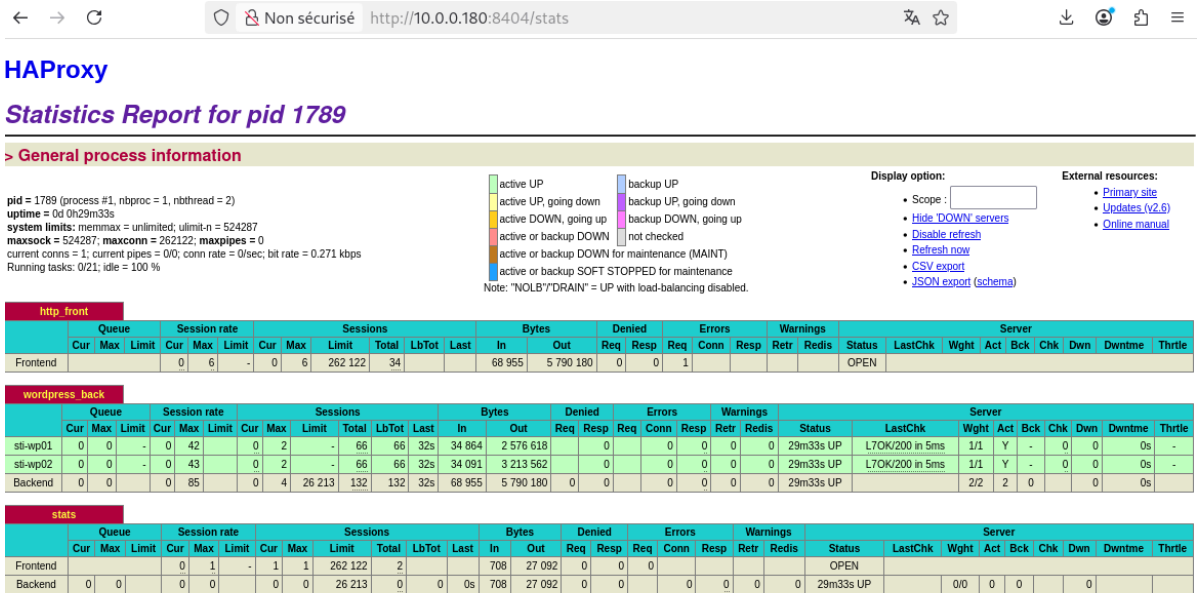


Figure 22 : Page d'état d'HAProxy (http://10.0.0.180:8404/stats) : sti-wp01 et sti-wp02 UP

```
user@debian:~/terraform/TD-INSA_Terraform/ansible$ curl -sI http://10.0.0.140/ |
head -1
HTTP/1.1 403 Forbidden
user@debian:~/terraform/TD-INSA_Terraform/ansible$
```

Figure 23 : Accès direct à sti-wp01 depuis un poste autre qu'HAProxy : 403 Forbidden

Simulation de panne : php8.2-fpm est arrêté sur sti-wp01, puis le site est interrogé à travers HAProxy.

```
root@sti-wp01:~# sudo systemctl stop php8.2-fpm
root@sti-wp01:~# systemctl is-active php8.2-fpm
inactive
root@sti-wp01:~#
```

Figure 24 : Panne simulée : arrêt de php8.2-fpm sur sti-wp01

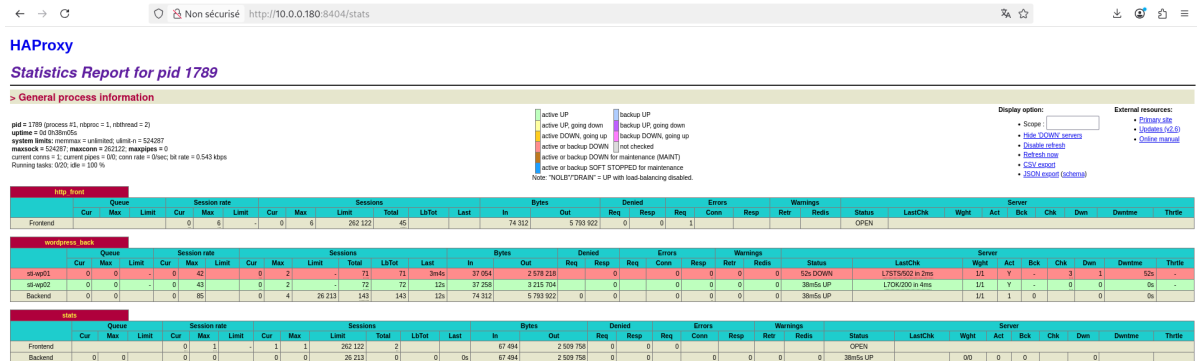


Figure 25 : Page d'état d'HAProxy pendant la panne : sti-wp01 DOWN (sonde /readyz en erreur 502), sti-wp02 UP

```
user@debian:~/terraform/TD-INSA_Terraform/ansible$ for i in 1 2 3 4 5 6; do curl
-sI http://10.0.0.180/ | grep -i '^x-served-by'; done
x-served-by: sti-wp02
x-served-by: sti-wp02
x-served-by: sti-wp02
x-served-by: sti-wp02
x-served-by: sti-wp02
x-served-by: sti-wp02
user@debian:~/terraform/TD-INSA_Terraform/ansible$ █
```

Figure 26 : Pendant la panne : les six requêtes sont servies par sti-wp02

Modification manuelle d'un nœud : Ansible l'affiche (--check --diff) sans rien changer, puis la corrige.

```
root@sti-wp01:~# echo '# modification manuelle' | sudo tee -a /etc/nginx/sites-a
vailable/wordpress
# modification manuelle
root@sti-wp01:~# █
```

Figure 27 : Modification sur sti-wp01.

```
user@debian:~/terraform/TD-INSA_Terraform$ cd ansible/
user@debian:~/terraform/TD-INSA_Terraform/ansible$ ansible-playbook site.yml --c
heck --diff --limit sti-wp01
[WARNING]: Deprecation warnings can be disabled by setting `deprecation_warnings
=False` in ansible.cfg.
[DEPRECATION WARNING]: community.mysql.mysql_query has been deprecated. Use ansi
ble.mysql.mysql_query instead. This feature will be removed from collection 'com
munity.mysql' version 6.0.0.
[DEPRECATION WARNING]: community.mysql.mysql_db has been deprecated. Use ansible
.mysql.mysql_db instead. This feature will be removed from collection 'community
.mysql' version 6.0.0.
[DEPRECATION WARNING]: community.mysql.mysql_user has been deprecated. Use ansib
le.mysql.mysql_user instead. This feature will be removed from collection 'commu
nity.mysql' version 6.0.0.

PLAY [Attendre que les VM soient prêtes] *****
```

Figure 28 : depuis ansible/ : ansible-playbook site.yml --check --diff --limit sti-wp01.

```
TASK [wordpress : Installer le site nginx] *****
--- before: /etc/nginx/sites-available/wordpress
+++ after: /home/user/.ansible/tmp/ansible-local-1400030gevl5j2/tmpgpf5xngso/nginx
wordpress.conf.j2
@@ -64,4 +64,3 @@
     location = /favicon.ico { log_not_found off; access_log off; }
     location = /robots.txt { log_not_found off; access_log off; }
 }
-# modification manuelle

changed: [sti-wp01]
```

Figure 29 : La différence apparaît.

```
user@debian:~/terraform/TD-INSA_Terraform/ansible$ ansible-playbook site.yml --limit sti-wp01
[WARNING]: Deprecation warnings can be disabled by setting `deprecation_warnings=False` in ansible.cfg.
[DEPRECATION WARNING]: community.mysql.mysql_query has been deprecated. Use ansible.mysql.mysql_query instead. This feature will be removed from collection 'community.mysql' version 6.0.0.
[DEPRECATION WARNING]: community.mysql.mysql_db has been deprecated. Use ansible.mysql.mysql_db instead. This feature will be removed from collection 'community.mysql' version 6.0.0.
[DEPRECATION WARNING]: community.mysql.mysql_user has been deprecated. Use ansible.mysql.mysql_user instead. This feature will be removed from collection 'community.mysql' version 6.0.0.

PLAY [Attendre que les VM soient prêtes] *****

TASK [Attendre la connexion SSH] *****
ok: [sti-wp01]
```

Figure 30 : ansible-playbook site.yml --limit sti-wp01 pour restaurer le fichier et recharger nginx.

```
TASK [wordpress : Extraire WordPress dans le dossier du site] *****
skipping: [sti-wp01]

TASK [wordpress : Supprimer le dossier temporaire] *****
skipping: [sti-wp01]

TASK [wordpress : Installer wp-config.php] *****
ok: [sti-wp01]

TASK [wordpress : Créer le dossier de la sonde] *****
ok: [sti-wp01]

TASK [wordpress : Installer la sonde readyz.php] *****
ok: [sti-wp01]

TASK [wordpress : Installer le site nginx] *****
changed: [sti-wp01]

TASK [wordpress : Activer le site] *****
ok: [sti-wp01]

TASK [wordpress : Retirer le site par défaut de nginx] *****
ok: [sti-wp01]

TASK [wordpress : Contrôler la configuration de nginx] *****
ok: [sti-wp01]

TASK [wordpress : Activer et démarrer nginx] *****
ok: [sti-wp01]

TASK [wordpress : Appliquer la configuration] *****

RUNNING HANDLER [wordpress : Recharger nginx] *****
changed: [sti-wp01]

TASK [wordpress : Vérifier que nginx répond (/healthz)] *****
ok: [sti-wp01]

TASK [wordpress : Vérifier que la base partagée est joignable (/readyz)] *****
ok: [sti-wp01]

TASK [wordpress : Supprimer l'ancien dossier de préparation (wp-config.php y était copié)] ***
ok: [sti-wp01]

PLAY [Configurer le load balancer] *****
skipping: no hosts matched

PLAY RECAP *****
sti-wp01      : ok=23   changed=2   unreachable=0   failed=0   skipped=4   rescued=0   ignored=0

user@debian:~/terraform/TD-INSA_Terraform/ansible$
```

Figure 31 : sti-wp01 restauré.

Services: Kea DHCP: Kea DHCPv4						
Paramètres Sous-réseaux Réservations Options Pairs de l'AP						
	Sous-réseau	Adresse IP	Adresse MAC	ID Client	Nom d'hôte	Description
☐	10.0.0.0/24	10.0.0.180	6e:6e:80:accc:e1		sti-lb01	Static IP for sti-lb01
☐	10.0.0.0/24	10.0.0.170	66:56:78:b7:16:13		sti-db01	Static IP for sti-db01
☐	10.0.0.0/24	10.0.0.160	86:f6:46:3a:ff:ce		sti-wp02	Static IP for sti-wp02
☐	10.0.0.0/24	10.0.0.140	fa:52:f9:c2:3e:bc		tf_cloned_debian	Static IP for tf_cloned_debian

Figure 32 : Réservations Kea DHCPv4 dans OPNsense : une IP fixe par VM (sti-wp01 garde son libellé d'origine, tf_cloned_debian)

```
macaddress.vm["sti-wp02"]: Creation complete after 0s [id=86:f6:46:3a:ff:ce]
macaddress.vm["sti-db01"]: Creation complete after 0s [id=66:56:78:b7:16:13]
macaddress.vm["sti-lb01"]: Creation complete after 0s [id=6e:6e:80:ac:cc:e1]
macaddress.vm["sti-wp01"]: Creation complete after 0s [id=fa:52:f9:c2:3e:bc]
```

Figure 33 : Adresses MAC créées par le provider ivoronin/macaddress pendant terraform apply : identiques à celles de la sortie vms et des réservations Kea

```
user@debian:~/terraform/TD-INSA_Terraform$ git check-ignore -v terraform.tfvars terraform.tfstate ansible/inventory
.ini ansible/group_vars/all/secrets.yml
.gitignore:7:terraform.tfvars terraform.tfvars
.gitignore:3:*.tfstate terraform.tfstate
.gitignore:10:ansible/inventory.ini ansible/inventory.ini
.gitignore:11:ansible/group_vars/ ansible/group_vars/all/secrets.yml
user@debian:~/terraform/TD-INSA_Terraform$ ls -l ansible/group_vars/all/
total 8
-rw----- 1 user user 112 1 oct. 17:54 secrets.yml
-rw-r--r-- 1 user user 336 1 oct. 17:54 terraform.yml
user@debian:~/terraform/TD-INSA_Terraform$
```

Figure 34 : Secrets exclus de git : règles du .gitignore et droits de secrets.yml.

```
Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

local_sensitive_file.ansible_secrets: Destroying... [id=031424a4116cecea5c869c0c7d46aa60785880c5]
local_file.ansible_inventory: Destroying... [id=25d4ed38b8412ae4952f7ec7650a64d217d7ecc2]
local_file.ansible_vars: Destroying... [id=5a3588191b8a91045e346d929ea64780607a5eb0]
local_sensitive_file.ansible_secrets: Destruction complete after 0s
local_file.ansible_inventory: Destruction complete after 0s
local_file.ansible_vars: Destruction complete after 0s
proxmox_vm_qemu.vm["sti-db01"]: Destroying... [id=pve/qemu/122]
proxmox_vm_qemu.vm["sti-wp02"]: Destroying... [id=pve/qemu/121]
proxmox_vm_qemu.vm["sti-lb01"]: Destroying... [id=pve/qemu/123]
proxmox_vm_qemu.vm["sti-wp01"]: Destroying... [id=pve/qemu/120]
proxmox_vm_qemu.vm["sti-db01"]: Destruction complete after 5s
proxmox_vm_qemu.vm["sti-wp01"]: Destruction complete after 8s
proxmox_vm_qemu.vm["sti-lb01"]: Still destroying... [id=pve/qemu/123, 00m10s elapsed]
proxmox_vm_qemu.vm["sti-wp02"]: Still destroying... [id=pve/qemu/121, 00m10s elapsed]
proxmox_vm_qemu.vm["sti-wp02"]: Destruction complete after 12s
proxmox_vm_qemu.vm["sti-lb01"]: Destruction complete after 16s
terraform_data.upload_snippet["sti-lb01"]: Destroying... [id=57f9d333-abf0-ab03-8507-a3d6df465bd9]
terraform_data.upload_snippet["sti-wp02"]: Destroying... [id=37f35f4d-850c-42cf-3411-293904a33b6d]
opnsense_kea_dhcpv4_reservation.vm["sti-wp02"]: Destroying... [id=731c8162-726a-44d3-88e8-4f472d0101a5]
opnsense_kea_dhcpv4_reservation.vm["sti-lb01"]: Destroying... [id=08991137-0f18-4b05-9b13-59db7ee54a54]
terraform_data.upload_snippet["sti-wp01"]: Destroying... [id=3083bca1-9b45-0f0a-6c40-7c09b1854cfd]
terraform_data.upload_snippet["sti-db01"]: Destroying... [id=0d37af7a-506a-2fad-ceac-7dd0b830ca86]
terraform_data.upload_snippet["sti-lb01"]: Destruction complete after 0s
opnsense_kea_dhcpv4_reservation.vm["sti-wp01"]: Destroying... [id=135e8080-498a-40d2-9d6c-3c32413c15ce]
opnsense_kea_dhcpv4_reservation.vm["sti-db01"]: Destroying... [id=59567764-736c-4bd3-ad5f-e5b7885cfbfe]
terraform_data.upload_snippet["sti-wp02"]: Destruction complete after 0s
terraform_data.upload_snippet["sti-wp01"]: Destruction complete after 0s
terraform_data.upload_snippet["sti-db01"]: Destruction complete after 0s
opnsense_kea_dhcpv4_reservation.vm["sti-lb01"]: Destruction complete after 1s
opnsense_kea_dhcpv4_reservation.vm["sti-wp02"]: Destruction complete after 1s
opnsense_kea_dhcpv4_reservation.vm["sti-wp01"]: Destruction complete after 1s
opnsense_kea_dhcpv4_reservation.vm["sti-db01"]: Destruction complete after 2s
macaddress.vm["sti-wp01"]: Destroying... [id=fa:52:f9:c2:3e:bc]
macaddress.vm["sti-lb01"]: Destroying... [id=6e:6e:80:ac:cc:e1]
macaddress.vm["sti-wp02"]: Destroying... [id=86:f6:46:3a:ff:ce]
macaddress.vm["sti-db01"]: Destroying... [id=66:56:78:b7:16:13]
macaddress.vm["sti-wp01"]: Destruction complete after 0s
macaddress.vm["sti-lb01"]: Destruction complete after 0s
macaddress.vm["sti-wp02"]: Destruction complete after 0s
macaddress.vm["sti-db01"]: Destruction complete after 0s

Destroy complete! Resources: 19 destroyed.
user@debian:~/terraform/TD-INSA_Terraform$
```

Figure 35 : terraform destroy.

5.4.5. Bilan de la plateforme

Le tableau récapitule ce qui fonctionne sur la plateforme déployée, la raison technique et les figures qui le montrent.

Ce qui fonctionne	Pourquoi	Voir
Le site est servi par les deux nœuds à tour de rôle	HAProxy répartit les requêtes en round-robin sur les VM du groupe wordpress, lues dans l'inventaire ; l'en-tête X-Served-By indique le nœud qui répond	Fig. 21, 22
Une panne d'un nœud ne coupe pas le site	la sonde /readyz (php-fpm et accès à MariaDB) est interrogée toutes les 2 s : après 3 échecs le nœud est retiré, après 2 succès il revient	Fig. 24, 25, 26
WordPress est installé et utilisable par un point d'entrée unique	le rôle wordpress installe nginx, php-fpm et WordPress sur chaque nœud ; HAProxy écoute sur 10.0.0.180 et relaie vers eux	Fig. 17, 18, 19
Les deux nœuds partagent le même contenu	tous deux se connectent à la même base MariaDB (sti-db01) avec un wp-config.php identique : même hôte de base, mêmes clés	Fig. 6, 9
La base n'est accessible qu'au réseau web	à distance, MariaDB n'accepte que le compte wordpress, limité à 10.0.0.0/24 ; les autres comptes sont locaux, et les comptes anonymes et la base test sont supprimés	Fig. 12
Les nœuds ne répondent qu'à HAProxy	nginx renvoie 403 à tout appelant autre qu'HAProxy (adresses lues dans l'inventaire) et ne croit X-Forwarded-For que s'il vient d'HAProxy	Fig. 23
Chaque VM garde la même adresse IP	l'adresse MAC est générée une seule fois (état Terraform) et associée à une IP fixe par une réservation Kea, créée avant le premier démarrage	Fig. 13, 32, 33
Le déploiement peut être rejoué sans effet	une seule table vms décrit les VM et alimente l'inventaire ; les rôles Ansible ne modifient que ce qui diffère, d'où changed=0 au second passage	Fig. 16
Une modification manuelle est détectée puis corrigée	--check --diff affiche l'écart avec les rôles sans rien modifier ; un passage normal rétablit l'état voulu	Fig. 27
Aucun secret n'est versionné	le mot de passe est dans un .tfvars ignoré par git ; Terraform écrit secrets.yml en 0600, ignoré lui aussi ; diff: false empêche son affichage dans les différences	Fig. 34

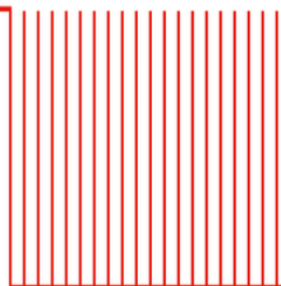
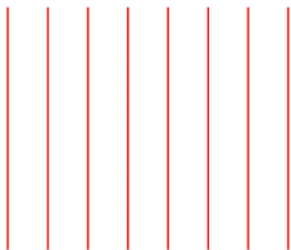
Le mot de passe figure aussi dans terraform.tfstate, comme tout attribut de ressource : ce fichier est lui aussi ignoré par git et ne doit pas être partagé.

Deux extensions complètent l'essentiel : les sondes de santé (/readyz) et le passage de 2 à N nœuds. Une ligne de plus dans vms suffit : terraform apply ajoute la VM à l'inventaire, ansible-playbook la configure et régénère le backend d'HAProxy. Le trafic reste en HTTP et la base n'est pas sauvegardée : ces deux points restent à traiter.

6. GitHub

Pour consulter les fichiers, et le cloner/manipuler:

[Voir Repo Github.](#)



INSA Centre Val de Loire

88 boulevard Lahitolle

CS 60013

18022 Bourges cedex

Tél : + 33 (0)2 48 48 40 00

www.insa-centrevaldeloire.fr



INSA INSTITUT NATIONAL
DES SCIENCES
APPLIQUÉES
CENTRE VAL DE LOIRE


**MINISTÈRE
DE L'ENSEIGNEMENT
SUPÉRIEUR
ET DE LA RECHERCHE**
*Liberté
Égalité
Fraternité*